



北方民族大学

本科毕业论文（设计）

题目：

基于 Vulkan 的实时光线追踪混合渲染器实现

学院名称： 计算机科学与工程学院

专业： 软件工程

学生姓名： 向晨

学号： 20221889

指导教师姓名： 韩强

企业指导教师： （无）

论文提交时间： 二〇二六年五月十五日

北方民族大学本科毕业论文（设计）诚信承诺书

学生姓名	向晨	年级	2022 级
所学专业	软件工程	学号	20221889
所在学院	计算机科学与工程学院		

学生承诺

本人郑重承诺和声明：

我承诺在毕业论文（设计）过程中严格遵守学校有关规定，在指导教师的安排与指导下独立完成所规定的毕业论文（设计）工作，决不弄虚作假，不请别人代做毕业论文（设计）或抄袭别人的成果。所撰写的毕业论文或毕业设计是在指导老师的指导下自主完成，文中所有引文或引用数据、图表均注解并说明来源，本人愿意为由此引起的后果承担责任。

学生（签名）：_____

2026 年 05 月 日

基于 Vulkan 的实时光线追踪混合渲染器实现

摘要

在现代实时计算机图形学中，传统的纯光栅化渲染在处理全局光照、精确物理软阴影与复杂镜面反射时存在固有的物理局限性，而离线级别的纯路径追踪又难以满足交互式应用的实时帧率需求。针对这一“画质与算力”的矛盾，本文基于现代显式图形 API Vulkan 1.3，设计并实现了一套工业级的高性能实时光线追踪混合渲染引擎。

本文的核心工作包括三个方面。首先，针对 Vulkan 繁琐的显存状态同步痛点，设计了数据驱动的 Render Graph 调度架构，通过有向无环图的拓扑分析实现了管线屏障的自动推导与显存复用优化。其次，构建了高效的混合光照管线，将几何光栅化与硬件光线追踪深度解耦。在延迟渲染 G-Buffer 的基础上，结合偏导数面法线偏移与 Ray Query 扩展实现了精准无伪影的软硬阴影，并利用 RT Pipeline 全面求解 Cook-Torrance 微面元模型，完成了基于物理（PBR）的多次镜面反射计算。最后，针对 1 SPP 极低采样率带来的蒙特卡洛高频噪声，完整集成了时空方差导向滤波（SVGF）降噪系统。通过几何运动矢量的时域历史重投影与动态方差引导的多轮 A-Trous 空域滤波，成功实现了高保真度的抗噪平滑处理。

性能测试与效果分析表明，本系统在 2K 原生分辨率下，能够以实时的性能开销（稳定大于 60 帧）输出平滑且物理正确的照片级渲染画面。本文的研究验证了以 Render Graph 为底座、结合光栅化、硬件光追与 SVGF 降噪的混合架构在现代底层图形引擎开发中的高效性与工程可行性。

关键词: Vulkan; 实时光线追踪; 混合渲染; Render Graph; 基于物理的渲染 (PBR); SVGF 降噪

Implementation of a Real-time Ray Tracing Hybrid Renderer Based on Vulkan

ABSTRACT

In modern real-time computer graphics, traditional pure rasterization rendering has inherent physical limitations when dealing with global illumination, precise physical soft shadows, and complex specular reflections, while offline-level pure path tracing is difficult to meet the real-time frame rate requirements of interactive applications. To address this contradiction between "image quality and computing power", this paper designs and implements an industrial-grade high-performance real-time ray tracing hybrid rendering engine based on the modern explicit graphics API Vulkan 1.3.

The core work of this paper includes three aspects. First, a data-driven Render Graph scheduling architecture was designed to address the tedious GPU memory state synchronization pain points of Vulkan. Through topological analysis of a directed acyclic graph, the automatic derivation of pipeline barriers and optimization of memory reuse were achieved. Second, an efficient hybrid lighting pipeline was constructed to deeply decouple geometric rasterization from hardware ray tracing. Based on deferred rendering G-Buffer, combined with derivative face normal offset and Ray Query extension, precise and artifact-free soft and hard shadows were implemented. Furthermore, the RT Pipeline was used to comprehensively solve the Cook-Torrance microfacet model, completing multi-bounce specular reflection calculations based on physics (PBR). Finally, addressing the high-frequency Monte Carlo noise caused by the extremely low sampling rate of 1 SPP, a complete Spatiotemporal Variance-Guided Filtering (SVGF) denoising system was integrated. High-fidelity anti-noise smoothing was successfully achieved through temporal history reprojection of geometric motion vectors and multi-round spatial A-Trous filtering guided by dynamic variance.

Performance tests and effectiveness analysis show that this system can output smooth and physically correct photo-realistic rendering results with real-time performance overhead (stable above 60 FPS) at 2K native resolution. This research verifies the efficiency and engineering

feasibility of a hybrid architecture based on Render Graph, combining rasterization, hardware ray tracing, and SVGF denoising in modern low-level graphics engine development.

Key words: Vulkan; Real-time Ray Tracing; Hybrid Rendering; Render Graph; Physically Based Rendering (PBR); SVGF Denoising

目 录

摘要	II
ABSTRACT	III
第一章 绪论	1
1.1 研究背景及意义	1
1.2 国内外研究现状	2
1.2.1 现代底层渲染引擎架构演进	2
1.2.2 实时光线追踪与混合渲染管线	2
1.2.3 实时信号重建与降噪算法现状	2
1.3 本文的主要工作与创新点	2
1.4 论文结构安排	3
第二章 实时混合渲染理论基础	4
2.1 基于物理的渲染 (PBR) 理论	4
2.1.1 微面元理论与 Cook-Torrance BRDF	4
2.1.2 渲染方程与蒙特卡洛积分	5
2.2 现代显式图形 API (Vulkan) 架构	5
2.2.1 显式资源管理与同步机制	5
2.2.2 Vulkan 现代特性: 动态渲染 (Dynamic Rendering)	5
2.3 硬件加速光线追踪原理	6
2.3.1 层次包围盒 (BVH) 加速结构	6
2.3.2 光线追踪管线 (RTP) 与光线查询 (Ray Query)	6
2.4 混合渲染 (Hybrid Rendering) 逻辑流程	7
2.4.1 延迟渲染与 G-Buffer 提取	7
2.4.2 基于空间与时域的信号重建	7
第三章 渲染器架构	8
3.1 渲染器概述	8
3.1.1 渲染介绍	8
3.2 系统架构设计	8

3.2.1	架构总体介绍	8
3.2.2	五层分层拓扑架构	9
3.3	模块化分层系统设计	10
3.3.1	Layer	10
3.3.1.1	Layer	10
3.3.1.2	LayerStack: 层级管理与堆栈结构	12
3.3.2	RenderGraph: 数据驱动的渲染管线调度	12
3.3.2.1	声明式依赖追踪	12
3.3.2.2	自动化资源生命周期	12
3.3.3	事件驱动与输入轮询	12
3.3.3.1	事件系统 (Event System)	13
3.3.3.2	输入轮询 (Input Polling)	13
3.4	程序入口与主循环	14
3.4.1	入口点设计	14
3.4.2	游戏循环 (Game Loop)	14
3.5	资源管理	15
3.5.1	资源对象抽象与封装 (Resource Abstraction)	15
3.5.2	中心化资源管理器 (ResourceManager)	15
3.5.3	基于 VMA 的显存池化管理	16
3.5.4	基于 Bindless 思想的全局描述符绑定	16
3.5.5	延迟销毁队列	16
3.6	引擎多线程架构与任务系统设计	17
3.6.1	多线程任务系统基础设施	17
3.6.2	后台异步资源处理流水线	17
3.6.3	前台并行指令录制架构	17
3.6.4	多线程环境下的同步机制	17
3.7	场景组织与空间加速算法	18
3.7.1	实体-组件式的场景结构	18
3.7.2	基于 AABB 的视锥体剔除 (Frustum Culling)	18
3.7.3	基于静态八叉树的空间划分 (Octree)	18
3.7.4	硬件加速的层次包围盒 (BVH)	18
3.8	交互式摄像机系统实现	19
3.8.1	摄像机原理: 操作倒转与观察空间	19
3.8.2	透视投影与裁剪空间	19
3.8.3	MVP 变换流水线	19

3.8.4 相机的交互实现	20
3.9 技术栈与构建方案	20
3.9.1 构建系统：CMake	20
3.9.2 核心依赖库清单	20
3.10 本章小结	20
第四章 渲染图与渲染管线实现	21
4.1 渲染图（Render Graph）调度系统设计	21
4.1.1 渲染图基本原理：自动化流水线工厂	21
4.1.2 架构设计动机：规避“同步地狱”	21
4.1.3 Chimera 引擎的系统实现	22
4.1.4 指令录制优化：基于静态多态的智能路由	24
4.1.4.1 特征探测与分发逻辑	24
4.1.4.2 架构优势	24
4.1.5 运行生命周期实录：Setup -> Compile -> Execute	25
4.1.5.1 Setup	25
4.1.5.2 Compile	25
4.1.5.3 Execute	25
4.1.6 工程优势总结	26
4.2 混合渲染管线的拓扑集成与数据流转	26
4.2.1 基于光栅化的几何信息采集	26
4.2.2 硬件加速光线追踪：按需查询与特征提取	28
4.2.3 信号合成与时域反馈环路	29
4.3 本章小结	29
第五章 后期处理与信号重建	30
5.1 信号采样与抗锯齿理论基础	30
5.1.1 光栅化中的离散采样与锯齿现象	30
5.1.2 光线追踪中的蒙特卡洛噪声	30
5.2 SVGF 时空方差引导降噪算法实现	30
5.2.1 光照分离与反照率解耦 (Albedo Demodulation)	31
5.2.2 时域滤波与方差估计 (Temporal Filtering & Variance)	32
5.2.3 空间滤波：联合双边 A-Trous 小波变换	33
5.3 时域抗锯齿（TAA）方案实现	34
5.3.1 TAA 基本原理	34
5.3.2 亚像素抖动与采样对齐 (Sub-pixel Jitter)	35

5.3.3	速度扩张与最近深度启发式搜索 (Velocity Dilation)	35
5.3.4	基于射线相交的历史帧裁剪 (Ray-Box Clipping)	35
5.3.5	渲染图驱动的历史反馈环路 (Temporal Feedback Loop)	36
5.3.6	自适应混合与结果合成	36
5.4	后期影像流水线与电影级合成	37
5.4.1	光学抖动还原与色调映射	37
5.4.2	曝光补偿与伽马校正 (Gamma Correction)	37
5.5	本章小结	37
第六章	系统测试与分析	38
6.1	实验环境与测试场景	38
6.2	混合管线画质定性分析	39
6.2.1	光栅化与光线追踪的光影正确性对比	39
6.2.2	SVGF 降噪模块的时空消融实验	40
6.2.3	TAA 亚像素抖动与边缘平滑分析	42
6.3	渲染性能与耗时定量分析	42
6.3.1	渲染阶段 GPU 耗时分布	42
6.3.2	性能结果总结与分析	42
第七章	总结与展望	44
7.1	全文工作总结	44
7.2	研究不足与未来展望	44

第一章 绪论

1.1 研究背景及意义

实时渲染（Real-time Rendering）是计算机图形学中极具挑战性的分支，其核心目标是在极短的时间窗口内（通常为 16.6ms 或 33.3ms 以内）生成高质量的图像信号，以维持人机交互的连贯性。与追求极致物理真实感但计算耗时极长的离线渲染（Offline Rendering）不同，实时渲染需要在有限的硬件资源与严苛的时间预算之间达成一种“物理正确性”与“计算吞吐量”的精妙平衡。

在过去的三十年中，基于扫描线填充的光栅化（Rasterization）技术凭借其硬件友好性与极高的执行效率，统治了实时图形领域。通过对几何图元的投影、插值与深度测试，光栅化管线能够支撑起现代大规模场景的实时呈现。然而，随着用户对视觉表现要求的不断提升，光栅化在处理全局光照、软阴影以及复杂反射等非局部光影特性时的局限性日益凸显。

2018 年，随着硬件光线追踪加速核心（RT Core）的问世及 Vulkan/DX12 等底层图形 API 的迭代，实时光线追踪（Real-time Ray Tracing）正式进入工程应用领域。它通过模拟光线在物理空间中的传播路径，从根本上解决了光栅化管线难以处理的可见性与遮挡难题。然而，即便在当前最先进的显卡架构下，要实现完全的实时路径追踪（Full Path Tracing）仍面临样本数极低（1 SPP）导致的噪声泛滥问题。因此，结合光栅化的高效几何提取与光线追踪的物理光影特性的“混合渲染（Hybrid Rendering）”架构，辅以高性能的时空重建算法，成为了当前学术界与工业界公认的演进方向。

1. **传统渲染技术的局限性：**探讨光栅化技术在处理全局光照、软阴影及复杂反射时的近似算法（如 SSR, Shadow Map）存在的走样与视觉断裂问题。
2. **硬件光线追踪的兴起：**分析 Vulkan DXR 扩展与 NVIDIA RTX 硬件加速为实时渲染带来的变革，以及“路径追踪”在实时端面临的低样本（1 SPP）噪点挑战。
3. **本课题的研究意义：**本文旨在通过构建一套基于数据驱动架构的混合渲染引擎，整合光栅化的速度优势与光线追踪的物理精确性，并引入时空滤波算法解决极低采样率下的图像重建问题，具有重要的工程应用价值。

1.2 国内外研究现状

1.2.1 现代底层渲染引擎架构演进

图形 API 从 OpenGL/DirectX 11 的状态机模式演进至 Vulkan/DirectX 12 的显式控制模式，对引擎架构提出了更高要求。

- **显式资源管理挑战：**论述手动管理 Pipeline Barrier、显存堆分配及命令缓冲提交的复杂性。
- **渲染图（Render Graph）架构：**调研 EA Frostbite 引擎首次提出的 FrameGraph 概念，分析虚幻引擎（RDG）与 Unity（SRP）如何利用有向无环图（DAG）实现自动资源生命周期管理与显存别名（Memory Aliasing）优化。

1.2.2 实时光线追踪与混合渲染管线

目前工业界公认的实时光追方案并非“全路径追踪”，而是基于混合渲染（Hybrid Rendering）的演进。

- **延迟渲染的基石：**回顾延迟着色（Deferred Shading）在处理大规模光源时的优势。
- **光追特性集成：**调研现代游戏引擎如何针对阴影（RTSH）、反射（RTR）及环境光遮蔽（RTAO）等特定环节精准触发光线探测，利用 Ray Query 技术在片段着色器中实现硬加速。

1.2.3 实时信号重建与降噪算法现状

在 1 SPP 的极端限制下，信号重建算法是决定最终画质的关键。

- **空间域滤波：**分析双边滤波（Bilateral Filter）及其变种在保边去噪方面的局限。
- **时域反馈（Temporal Feedback）：**论述时空抗锯齿（TAA）与历史信息重投影（Reprojection）在平滑信号中的核心地位。
- **SVGF 理论体系：**重点分析 2017 年提出的时空方差引导滤波（SVGF）如何通过解耦反照率（Albedo）与光照信号，利用时域梯度估计动态调整滤波器核大小。

1.3 本文的主要工作与创新点

本文从零构建了一套名为“Chimera”的高性能实时混合渲染引擎，主要工作如下：

1. **设计并实现了基于 DAG 的声明式 Render Graph：**通过拓扑排序与资源引用计数，实现了 Vulkan 内存屏障的自动化插入与 Swapchain 图像布局的智能转换，极大降低了管线切换的复杂度。
2. **构建了物理正确的混合渲染管线：**基于 Vulkan 1.3 动态渲染特性，实现了结合 GLTF 2.0 模型加载、PBR 材质模型、Ray Traced 软阴影与镜面反射的高效率管线。

3. **落地了全套时空重建与后处理系统：**实现了 SVGF 降噪算法及抖动补偿的 TAA 方案，引入了基于颜色直方图裁剪（Color Clipping）的重投影优化，解决了动量失效导致的残影问题。

1.4 论文结构安排

本文共分为七章，具体内容安排如下：

- **第一章绪论：**阐述研究背景、意义及国内外研究现状。
- **第二章实时混合渲染理论基础：**介绍 PBR 物理渲染模型（BRDF）、Vulkan 渲染机制及硬件光线追踪（AS/RT Pipeline）原理。
- **第三章引擎架构设计与底层实现：**论述引擎的分层模块化设计、Vulkan 初始化流程及 EditorCamera 交互系统的构建。
- **第四章渲染图调度与管线集成：**详细解析 Render Graph 资源依赖推导、G-Buffer 布局设计及光追/光栅化混合调度。
- **第五章信号降噪与画质增强算法实现：**深入探讨 SVGF 时空滤波细节、TAA 抖动策略、Bloom 泛光及 Tone Mapping 后处理。
- **第六章系统实验与分析：**基于 DamagedHelmet 等工业级 PBR 模型展示渲染效果，并针对不同降噪配置进行性能消融实验（Ablation Study）。
- **第七章总结与展望：**总结本文成果，指出当前系统在多弹射 GI、模型简化等方面的局限及改进方向。

第二章 实时混合渲染理论基础

本章将阐述构建实时混合渲染引擎所需的理论基石。首先介绍基于物理的渲染（PBR）数学模型，随后探讨现代 Vulkan API 的显式驱动机制，最后详细解析硬件加速光线追踪的底层原理，为后续章节中渲染图架构与降噪算法的实现奠定理论基础。

2.1 基于物理的渲染（PBR）理论

2.1.1 微面元理论与 Cook-Torrance BRDF

为了模拟真实世界的材质表现，通常采用微面元模型（Microfacet Theory），假设表面由无数微小的镜面组成。微面曲面模型通常由给出微面法线 n_f 关于曲面法线 n 的分布函数来描述，如图 2.1 所示。

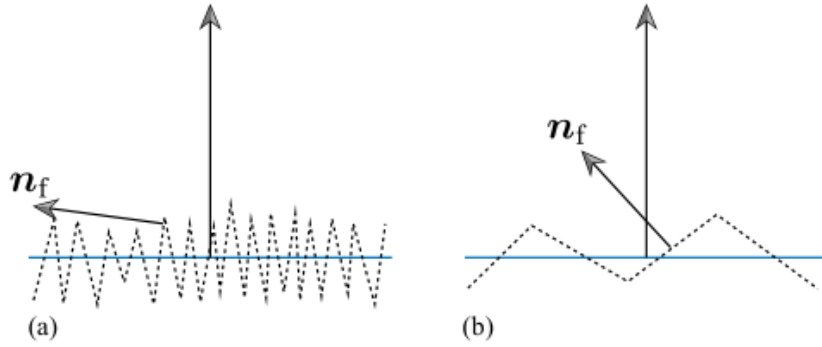


图 2.1 微面元模型示意图：(a) 微面法线变化越大，表面越粗糙；(b) 光滑表面的微面法线变化相对较小

其核心的双向反射分布函数（BRDF）采用 Cook-Torrance 模型：

$$f_r(p, \omega_i, \omega_o) = k_d \frac{c}{\pi} + k_s \frac{D(h)F(v, h)G(l, v, h)}{4(n \cdot l)(n \cdot v)} \quad (2.1)$$

其中， k_d 和 k_s 分别代表漫反射和镜面反射的能量比例，满足能量守恒定律。 $D(h)$ 为 Trowbridge-Reitz GGX 分布函数， F 采用 Schlick 近似菲涅尔项， G 则基于 Smith 模型描述表面的几何遮蔽与阴影效应。

2.1.2 渲染方程与蒙特卡洛积分

场景中任意点的出射辐亮度 L_o 由 Kajiya 提出的渲染方程（Rendering Equation）描述：

$$L_o(p, \omega_o) = L_e(p, \omega_o) + \int_{\Omega} f_r(p, \omega_i, \omega_o) L_i(p, \omega_i) (n \cdot \omega_i) d\omega_i \quad (2.2)$$

该方程指出，某点的出射光是其自身发光 L_e 与半球范围内入射光 L_i 经过表面反射后的总和。在实时混合渲染中，由于积分项无法解析求得，通常采用蒙特卡洛积分进行离散采样估算：

$$L_o(p, \omega_o) \approx \frac{1}{N} \sum_{i=1}^N \frac{f_r(p, \omega_i, \omega_o) L_i(p, \omega_i) (n \cdot \omega_i)}{p(\omega_i)} \quad (2.3)$$

其中 $p(\omega_i)$ 是对应的概率密度函数（PDF）。在 1 SPP（每像素一采样）的严苛限制下，这种随机采样会导致严重的噪声，这是后续 SVGF 降噪算法需要解决的核心矛盾。

2.2 现代显式图形 API（Vulkan）架构

2.2.1 显式资源管理与同步机制

不同于传统的图形 API（如 OpenGL）采用隐式状态机管理，Vulkan 强制要求开发者对底层硬件行为进行显式驱动：

- **显存生命周期管理**：论述显存堆（Memory Heap）的手动分配流程。通过 `vkBindBufferMemory` 实现显存地址与资源对象的关联，极大地提高了显存利用率。
- **屏障与同步**：Vulkan 1.3 的 `Synchronization2` 机制允许通过 `VkImageMemoryBarrier2` 显式描述 GPU 上的执行依赖（Stage Mask）与内存访问（Access Mask），有效解决了“写后读（RAW）”等数据竞争问题。

2.2.2 Vulkan 现代特性：动态渲染（Dynamic Rendering）

传统 Vulkan 加速管线的构建需要繁琐的渲染通道（Render Pass）与帧缓冲（Framebuffer）对象，这不仅增加了代码冗余，还限制了渲染图（Render Graph）在动态切换 Pass 时的灵活性。

本文全面采用了 Vulkan 1.3 的动态渲染特性（`KHR_dynamic_rendering`）：

- **架构简化**：通过 `vkCmdBeginRendering` 替代了复杂的渲染通道对象创建，允许在执行时动态指定颜色、深度附件的视图（ImageView）与格式。
- **Render Graph 兼容性**：该特性使得渲染图中的每个虚拟节点（Virtual Node）可以更加轻量地映射到物理资源，消除了由于 Render Pass 布局不匹配导致的管线重编译开销。

2.3 硬件加速光线追踪原理

2.3.1 层次包围盒（BVH）加速结构

为了实现高效的射线-几何体求交，现代 GPU 引入了双层加速结构（Acceleration Structures），其层级拓扑关系如图 2.2 所示。

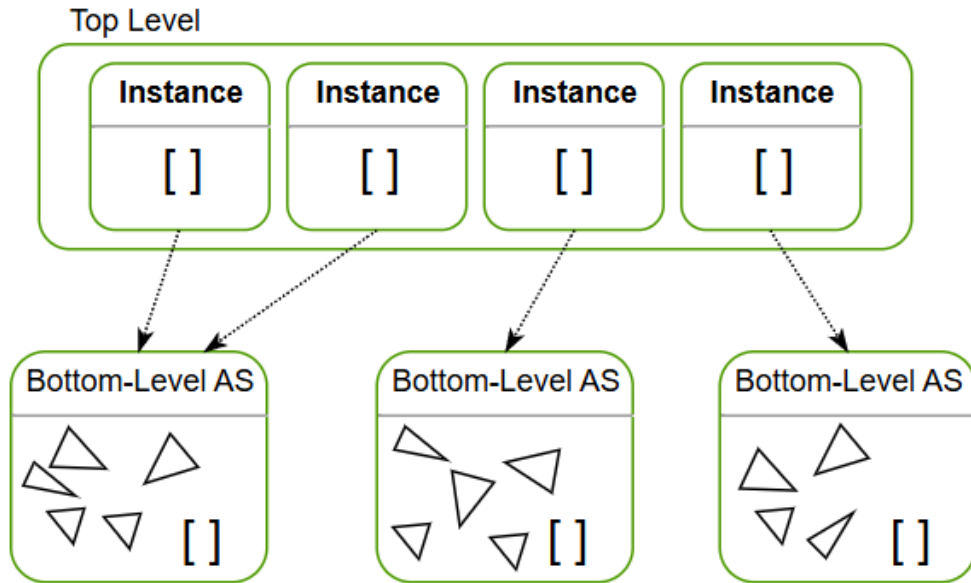


图 2.2 硬件加速的光线追踪双层包围盒（BVH）架构示意图

- **底层加速结构（BLAS）：**存储原始顶点数据与索引，负责局部的交点计算。驱动程序会根据几何体的拓扑结构自动构建最优的二叉树。
- **顶层加速结构（TLAS）：**存储多个 BLAS 的实例引用（Instance）及其对应的 4×4 世界变换矩阵。这种解耦设计使得场景中物体的平移、旋转仅需重构轻量级的 TLAS，而无需改动庞大的顶点数据，从而支持了动态场景的实时渲染。

2.3.2 光线追踪管线（RTP）与光线查询（Ray Query）

Vulkan

- **RT Pipeline：**通过 RayGen、Miss、ClosestHit 等专用着色器阶段实现复杂的路径追踪逻辑。
- **Ray Query：**允许在任意着色器阶段（如 Fragment Shader）发起射线探测，适用于混合渲染中的硬阴影与环境光遮蔽计算。

2.4 混合渲染（Hybrid Rendering）逻辑流程

2.4.1 延迟渲染与 G-Buffer 提取

混合渲染管线的第一阶段采用光栅化方式生成几何缓冲区（G-Buffer），提取场景的深度（Depth）、法线（Normal）、反照率（Albedo）及材质参数。这种方式减少了后续光线追踪管线的计算量，为后续光线追踪管线的计算提供了必要的信息。

2.4.2 基于空间与时域的信号重建

由于 1 SPP 的光追结果存在严重的统计噪声，必须结合 G-Buffer 提供的辅助信息进行信号重建。本文将利用 G-Buffer 中的运动矢量（Motion Vector）进行时域重投影，并通过空间域滤波器实现最终的平滑输出。

第三章 渲染器架构

本章将详细介绍 Chimera 渲染引擎的整体架构。首先从宏观视角阐述渲染器的基本定义及其在图形学系统中的位置，随后引出本项目参考的分层拓扑架构设计。本章还将深入探讨模块化分层系统、场景管理、资源调度以及交互式摄像机系统的实现细节，并总结项目所采用的技术栈。

3.1 渲染器概述

3.1.1 渲染介绍

渲染（Rendering）是三维计算机图形学中用软件在平面上从三维模型、场景生成二维图像的过程，计算机需要对三维模型或场景进行处理，包括建模、纹理映射、光照计算等一系列计算，最终生成一张二维图像。

渲染器（Renderer）是游戏引擎中最核心的子系统之一。它负责将抽象的游戏世界数据（如模型顶点、纹理采样器、光照参数等）转换为最终呈现在屏幕上的图像。它将游戏世界的逻辑状态翻译为 GPU 可执行的底层图形操作。

在一个完整的游戏引擎，渲染器只是庞大系统架构中的很小的一部分，相比于游戏引擎，渲染器不用实现 Gameplay 层的以及与图形学研究无关的物理引擎和音频系统等，但在底层架构上，渲染器与现代游戏引擎有着极高的相似度。一个完备的渲染器依然需要具备模型加载、资源管理、事件响应等基础功能。

3.2 系统架构设计

3.2.1 架构总体介绍

在搭建渲染器时，我参考了 Walnut，一个设计明晰，轻便的渲染引擎，在此基础上实现了 RenderGraph 管理的渲染层架构。

在工程实现上，把具体的渲染器库核心编译为一个静态库，实际的界面，渲染在 Sandbox 调用

- **引擎核心**：被编译为一个静态库，封装了和抽象图形 API 调用和渲染调度逻辑，让渲染和用户的代码隔离。
- **Sandbox**：作为一个独立的的可执行项目，通过调用引擎库提供的 API 来编写具体的渲染指令，不需要知道底层的渲染逻辑。

这样能给用户提供一个不受干扰的开发环境。在 Sandbox 中，用户专心写渲染代码，而无需关心底层的渲染实现。

3.2.2 五层分层拓扑架构

《游戏引擎架构》这本书里总结了游戏引擎的架构的分层，每层的功能职责。为了提高系统的可扩展性并降低模块间的耦合度，本项目参考 Walnut 采用了清晰的分层拓扑架构。整个引擎从顶层应用到底层硬件共划分为五个核心逻辑层，如图 3.1 所示。

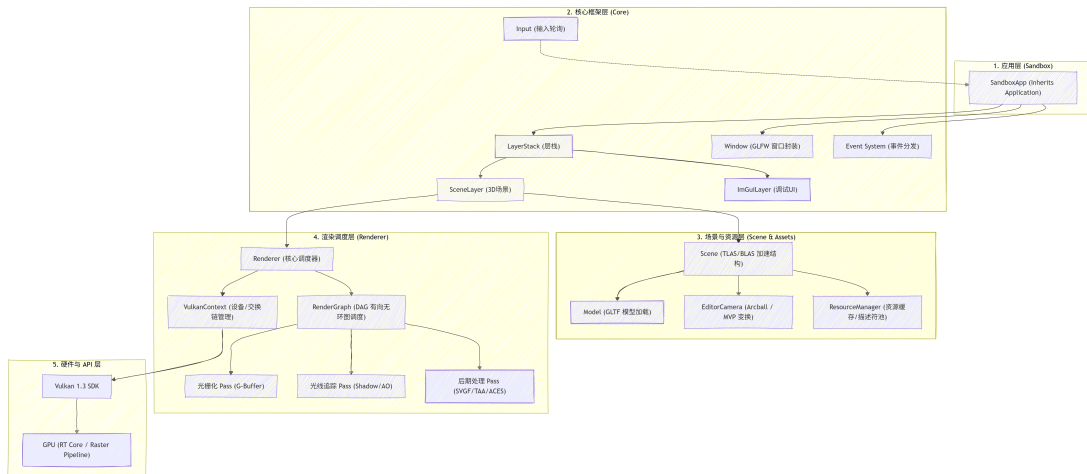


图 3.1 Chimera 引擎五层分层拓扑架构图

1. **应用层 (Sandbox Layer):** 具体的沙盒应用入口。通过继承 Application 基类并挂载自定义的 Layer，开发者可以快速构建不同的渲染场景。
2. **核心框架层 (Core Layer):**
 - **LayerStack:** 支持“插拔式”的模块管理，如处理 3D 渲染的 SceneLayer 和处理 UI 的 ImGuiLayer。
 - **事件系统:** 采用“从顶到底”的事件拦截机制，确保 UI 操作不会误触发场景交互。
 - **日志系统:** 集成 spdlog，能在开发过程中让用快速定位错误。
3. **场景与资源层 (Scene & Assets Layer):**
 - **加速结构 (AS):** 负责实时更新光线追踪所需的 TLAS（顶层加速结构）和 BLAS（底层加速结构）。
 - **ResourceManager:** 资源管理类单例，利用延迟销毁队列确保在 GPU 繁忙时不会误删资源。
4. **渲染调度层 (Renderer Layer):**

- **RenderGraph:** 核心调度中枢。不仅通过 DAG（有向无环图）自动推断 Pass 间的依赖关系，还实现了多线程就绪的层级化调度架构（Parallel Leveling），支持在 CPU 侧进行并行的指令录制。
- **资源抽象:** 封装了纹理（Image）、缓冲区（Buffer）及着色器（Shader）等资源对象。

5. **平台与硬件层 (Platform Layer):** 直接与操作系统交互。

- **Vulkan 环境:** 利用 volk 动态加载 API 入口，并负责逻辑设备、交换链及验证层的 Vulkan 基本配置。
- **窗口管理:** 通过 GLFW 捕获原生输入事件并转发至核心层。

3.3 模块化分层系统设计

介绍渲染器部分模块的设计。

3.3.1 Layer

3.3.1.1 Layer

Layer（层）是一个高度抽象的逻辑概念，它是引擎的一部分，它用于呈现某些东西的内容并接收事件。在 Chimera 中，Layer 类层次结构如图 3.2 所示：

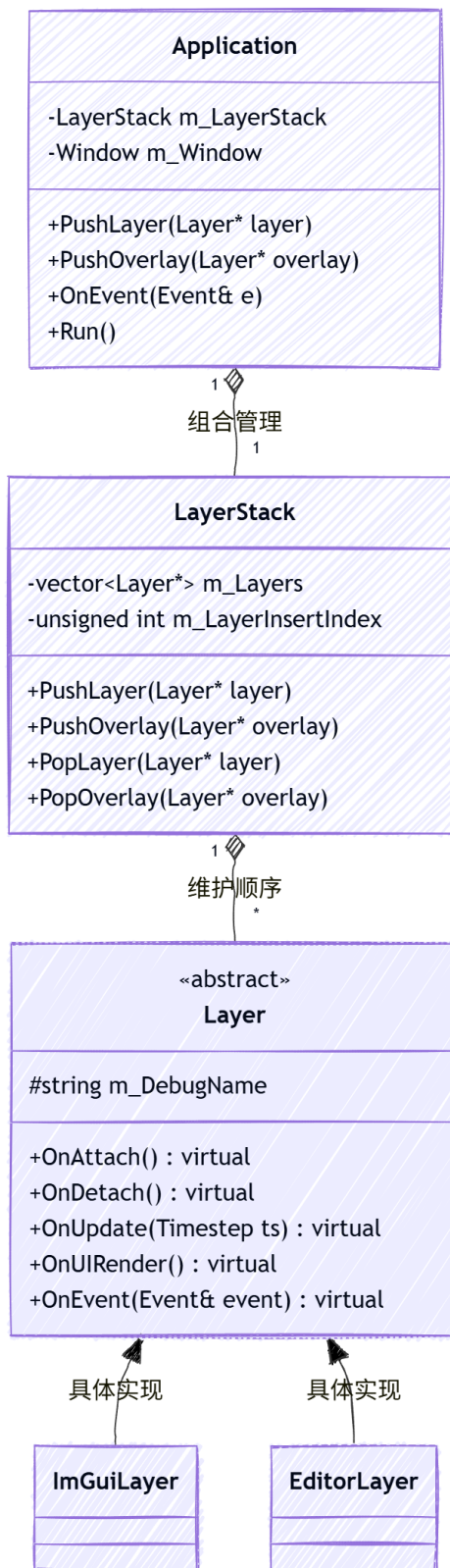


图 3.2 Layer 抽象基类与具体实现类关系图

- **OnAttach / OnDetach:** 当层被推入或移出栈时触发,用于资源的初始化与释放。
- **OnUpdate:** 每帧执行一次,用于处理该层特有的逻辑更新(如相机移动、场景渲染)。
- **OnImGuiRender:** 专门用于提交 ImGui 绘图指令,将调试 UI 与核心场景渲染逻辑分离。
- **OnEvent:** 接收来自应用层的分发事件,并根据逻辑决定是否处理和拦截。

在本项目中, Layer 用户可以在自己的项目里继承 Layer 类,搭建用于测试的窗口并实现自己的逻辑,所做的修改不会影响到核心逻辑。

3.3.1.2 LayerStack: 层级管理与堆栈结构

为了确保类的功能完整和正常的生命期,引擎实现了 LayerStack 类。它在内部维护了一个包含所有类的容器,按顺序执行类的接口

- **普通层 (Layer):** 按照添加顺序存储在容器的前半部分。例如,底层的环境背景层先添加,上层的模型层后添加。
- **覆盖层 (Overlay):** 始终被维护在容器的最末端。它们通常具有最高优先级,用于显示始终置顶的调试信息、性能 Overlay 或全局弹出通知。

3.3.2 RenderGraph: 数据驱动的渲染管线调度

RenderGraph 是 Chimera 引擎的“大脑”,它负责将零散的渲染 Pass 组织成一个高效的、自动同步的执行流水线。

3.3.2.1 声明式依赖追踪

与传统引擎手动管理 Vulkan 屏障不同,Chimera 采用了声明式接口。Pass 开发者只需通过 PassBuilder 声明资源的读写关系(如 Read("Albedo") 或 Write("ShadowMap")),由渲染图自动推导底层的 vkCmdPipelineBarrier2。这种设计极大降低了 Vulkan 开发中同步错误的概率。

3.3.2.2 自动化资源生命周期

通过分析资源的首次与末次使用点,RenderGraph 能够自动管理物理显存的分配与回收。这不仅简化了开发流程,还允许不同时重叠的逻辑资源复用同一块物理显存,有效降低了系统的显存峰值占用。

3.3.3 事件驱动与输入轮询

为了实现平滑且高性能的交互响应,本项目设计了一套结合“事件驱动(Event-Driven)”与“输入轮询(Input Polling)”的混合交互架构。

3.3.3.1 事件系统（Event System）

事件系统主要用于处理“一次性”的动作，如窗口缩放、鼠标点击或按键按下。所有事件类均继承自 `Event` 基类，并按功能划分为 `KeyEvent`、`MouseEvent` 与 `ApplicationEvent` 等类别。

事件处理的核心中枢是 `EventDispatcher`（事件分发器），匹配输入和对应的回调函数。

- **按需拦截：**分发器通过 `Dispatch<T>` 方法尝试将通用事件匹配为特定类型（如 `MouseScrolledEvent`）。
- **消耗机制：**与之绑定的回调函数会返回一个布尔值。如果返回 `true`，则表示该事件已被当前 `Layer`，后面层无需处理。

`Layer` 渲染和调事件的处理逻辑相反。渲染从下往上绘制，事件从上往下处理。比如我选中的是最上面的调试按键，那么就不会由下一层处理，而渲染会从下往上绘制。

1. **渲染流（从底向上，Bottom-to-Top）：**在 `OnUpdate` 阶段，引擎从 `LayerStack` 的最底层（索引 0）开始遍历。这样做可以确保背景先绘制，模型次之，最后是覆盖在最上方的 UI 界面。这种正向遍历符合图形学的深度覆盖逻辑。
2. **事件流（从顶向下，Top-to-Bottom）：**当鼠标点击或按键触发时，事件分发器会从最顶层的 `Overlay` 开始倒序遍历（从栈顶到栈底）。

这种“从顶向下”的事件传递机制实现了**事件拦截（Event Interception）**。例如，当用户点击 `ImGui` 面板上的一个按钮时，顶层的 `ImGuiLayer` 会捕获该点击事件并将其标记为“已处理（Handled）”。此时，下层的 `SceneLayer` 将不再收到该事件，从而有效避免了“穿透点击”现象——即在操作 UI 菜单时意外触发了场景中物体的选择或移动。

3.3.3.2 输入轮询（Input Polling）

虽然事件系统能够精准捕获动作，但却是一次性的，对于需要“连续平滑响应”的交互（如 WASD 相机漫游）等每帧主动查询当前的键盘或鼠标状态，就需要引入输入轮询。

通过静态类 `Input`，系统可以直接查询当前硬件的瞬时状态，其类结构如图 3.3 所示：

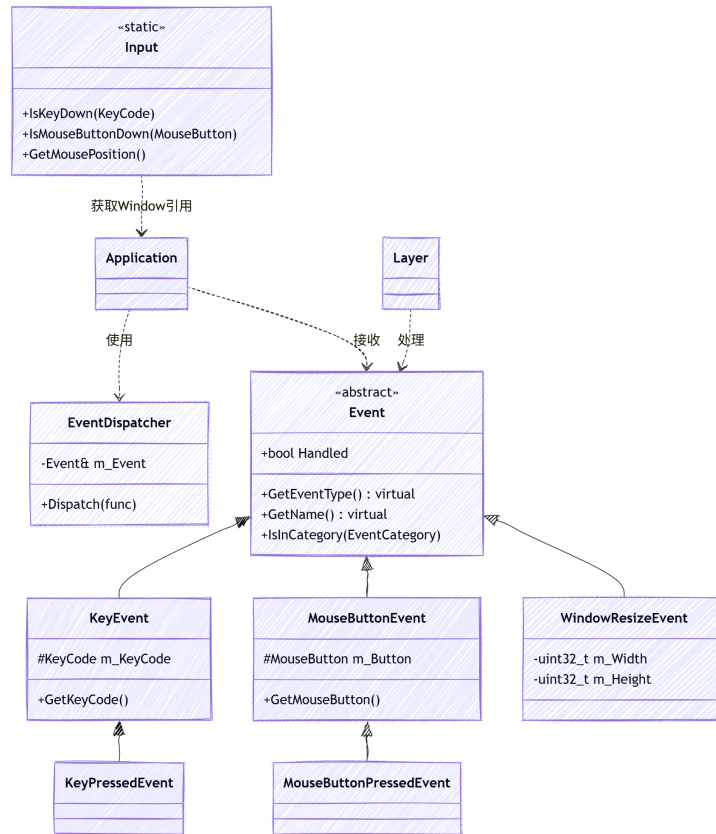


图 3.3 Input 静态类及其硬件接口封装图

- **主动查询:**在 OnUpdate 逻辑中,代码可以直接调用 `Input::IsKeyPressed(Key::W)`。
- **状态同步:** 该接口本质上是对底层 GLFW 输入状态的封装,能够以与帧率同步的频率获取输入,从而实现无延迟的平滑移动体验。

事件系统负责“决策与切换”(如切换渲染路径、开关 TAA),而输入轮询负责“连续交互与操作”。

3.4 程序入口与主循环

3.4.1 入口点设计

本项目使用了游戏开发中经典的游戏循环模式模式。

3.4.2 游戏循环 (Game Loop)

在 `Application::Run()` 方法中,引擎维持着一个高效的死循环。每一帧的执行下面操作:

1. **获取步长:** 计算两帧之间的时间差 (Timestep),用于物理平滑交互。
2. **事件轮询:** 通过获取最新的鼠标和键盘状态。

3. **层级更新**: 遍历 LayerStack, 执行每一层的 OnUpdate 逻辑。
4. **渲染提交**: 调用 Renderer 提交当前帧的绘制命令缓冲。
5. **UI 渲染**: 在场景渲染完成后, 叠加 ImGui 调试界面。

3.5 资源管理

Vulkan 的复杂度源于其对资源生命周期、内存分配以及同步机制的显式控制。在 Vulkan 这种显式图形 API 中, 资源管理是渲染器开发中最具挑战性的模块之一。为了确保系统的健壮性并降低 CPU 端的调度开销, Chimera 实现了一套高度抽象的“中心化资源调度系统”, 通过单例模式的 ResourceManager 统一管控所有图形资源。

3.5.1 资源对象抽象与封装 (Resource Abstraction)

为了规避原始 Vulkan 句柄带来的管理负担, 引擎将琐碎的 Vulkan 底层对象 (如 VkBuffer、VkImage 等) 封装为高级 C++ 类, 包括 Buffer、Image 与 Material 类。

其核心设计在于:

- **RAII 生命周期管理**: 利用 C++ 的构造与析构函数实现资源的自动分配与释放。在对象析构时, 系统会自动调用 Vulkan 的销毁函数并协同 VMA (Vulkan Memory Allocator) 释放关联的显存空间, 从根本上杜绝了显存泄露的风险。
- **内存抽象与便捷访问**: 封装类隐藏了复杂的内存映射 (Memory Mapping) 与对齐逻辑。例如, Buffer 类提供了直观的 UploadData() 接口, 内部自动处理暂存缓冲区 (Staging Buffer) 的中转或直接内存映射, 极大简化了宿主机 (Host) 与设备 (Device) 之间的数据通信。
- **材质数据驱动设计**: Material 类封装了基于 PBR 定义的材质参数。通过内部维护的“脏标记 (Dirty Flag)”机制, 系统仅在参数发生变化时才通过 ResourceManager 触发向 GPU 端的同步, 有效减少了冗余的内存拷贝。

3.5.2 中心化资源管理器 (ResourceManager)

引擎引入了单例模式的 ResourceManager 作为系统的核心资源枢纽, 负责资源的全局生命周期调度与运行时管理。

该管理器的核心职责体现如下:

- **统一调度与池化存储**: ResourceManager 内部维护着连续的资源存储池。即便底层资源由于加载或动态更新而发生变动, 上层逻辑层依然可以通过稳定的接口访问资源, 实现了逻辑层与底层物理实现的完全解耦。
- **延迟释放队列 (Resource Free Queue)**: 考虑到 GPU 指令流的异步执行特性, 管

理器引入了延迟释放机制。当资源不再被引用时，其销毁操作会被推入一个按帧排序的队列，确保资源仅在确认 GPU 完成相关指令集后才进行物理释放，保证了多帧并行下的执行安全。

- **全局描述符管理与同步：**管理器负责维护场景级别的全局描述符集（Global Descriptor Sets）。它自动将纹理数组、材质参数缓冲区等映射到特定的绑定点（Binding），支持“无绑定纹理（Bindless Texture）”技术，允许着色器通过索引直接访问海量纹理资源，显著降低了绘制调用（Draw Call）之间的状态切换开销。

3.5.3 基于 VMA 的显存池化管理

在底层显存分配上，本项目集成了 AMD 开发的 Vulkan Memory Allocator（VMA）。由于 Vulkan 物理设备对内存分配次数有严格限制，直接为每个纹理调用 `vkAllocateMemory` 会迅速耗尽系统资源。VMA 通过在大块预分配内存（Heap）上进行子分配（Sub-allocation）和碎片整理。Chimera 将其封装在 `Buffer` 与 `Image` 类中，极大地简化了 Staging Buffer 的创建与数据同步流程。

3.5.4 基于 Bindless 思想的全局描述符绑定

为了支持百万级面片与海量纹理的实时混合渲染，Chimera 采用了现代化的 `**Bindless**` 资源绑定架构：

1. **材质存储缓冲区（SSBO）：**`ResourceManager` 会将场景中所有材质的 PBR 参数（颜色、金属性、纹理索引等）打包成一个连续的 `GpuMaterial` 数组，并通过 `SyncMaterialsToGPU` 一次性上传至全局存储缓冲区。
2. **全局纹理数组：**引擎维护了一个巨大的描述符数组（Descriptor Array），将场景中所有的纹理视图（`ImageView`）一次性绑定至全局 Descriptor Set。

在着色器中，程序不再需要频繁切换纹理绑定点，而是通过材质结构体中存储的索引值，直接在全局数组中访问对应的纹理。这种设计彻底消除了光栅化时代“一个物体一个 Draw Call”的性能瓶颈。

3.5.5 延迟销毁队列

Vulkan 严禁在 GPU 仍在执行渲染指令时销毁其引用的资源。为了解决 CPU-GPU 异步导致的资源竞争问题，Chimera 实现了一套延迟销毁机制（Deletion Queue）：当用户请求释放某个资源时，`ResourceManager` 并不会立即调用销毁函数，而是将其推入队列并标记当前的帧序号。系统会等待两到三帧（对应双重或三重缓冲的完整循环）后，确保 GPU 已经彻底释放了对该资源的引用，才会执行最终的物理清理。这一机制是引

擎能够稳定运行、不发生显存溢出的关键保障。

3.6 引擎多线程架构与任务系统设计

为了充分利用现代多核 CPU 的计算性能，Chimera 引擎构建了一套贯穿“后台资源处理”与“前台指令录制”的全方位多线程架构。

3.6.1 多线程任务系统基础设施

引擎在初始化阶段会创建一个常驻线程池（Thread Pool），其工作线程数量根据宿主的硬件核心数动态分配。通过封装 `std::future` 机制，系统提供了一个轻量级的任务调度接口，允许主线程以非阻塞的方式分发耗时的计算任务。

3.6.2 后台异步资源处理流水线

在处理大规模复杂场景时，同步加载会导致主线程出现明显的卡顿。Chimera 实现了一套四阶段异步加载流程：

1. **IO 与解析阶段：**在后台线程调用 `cgltf` 读取文件。
2. **并行解码阶段：**利用多核优势，将数十张贴图分发至线程池并行解码。
3. **异步 GPU 上传：**在后台录制上传指令并提交。
4. **延迟集成阶段：**主线程每帧轮询，准备就绪后通过廉价的指针交换挂载至场景树。

3.6.3 前台并行指令录制架构

不同于传统的线性渲染调度，Chimera 的 `RenderGraph` 具备向多线程录制演进的层级化架构。

基于 DAG 的层级化调度 (Parallel Leveling)：在渲染图编译阶段，系统通过分析资源的读写依赖（Last Writer），自动构建有向无环图。系统利用分层算法将互不冲突的 Pass 分配到相同的“并行层级”中。例如，直接光阴影图的生成与 G-Buffer 的几何采集由于不存在数据竞争，被分配在同一层级。

层级化同步与屏障批处理：在执行阶段，系统采用“主线程同步，多线程录制”策略。主线程在每一层级开始前，统一提交该层所有 Pass 所需的资源布局转换屏障。这种设计极大压低了 CPU 侧的 API 调用负载，并为未来接入 Secondary Command Buffer 真正的并行录制奠定了逻辑基础。

3.6.4 多线程环境下的同步机制

由于 Vulkan 的部分对象（如 `VkQueue`）是非线程安全的，系统在 `VulkanContext` 层引入了全局队列互斥锁，确保了后台资源上传与前台渲染指令提交之间的外部同步。

3.7 场景组织与空间加速算法

如果说资源系统提供了渲染的“砖块”，那么场景系统则定义了“建筑的拓扑结构”。在 Chimera 引擎中，为了兼顾传统光栅化管线的提交效率与实时光线追踪的遍历性能，本文实现了一套“CPU 空间划分 + GPU 硬件加速”的双层加速架构。

3.7.1 实体-组件式的场景结构

为了兼顾开发灵活性与运行效率，Chimera 的场景层采用了轻量级的组件化设计：

- **Entity（实体）**：场景中的基本逻辑单元，持有全局唯一的 ID 与关联的组件索引。
- **TransformComponent**：维护物体的 4x4 TRS 变换矩阵，并追踪前一帧变换数据，为 TAA 提供了关键的运动矢量（Motion Vector）。
- **MeshComponent**：作为几何体载体，持有关联的 Model 引用，实现了几何数据与逻辑实体的解耦。

3.7.2 基于 AABB 的视锥体剔除 (Frustum Culling)

在光栅化阶段，盲目提交视野外的物体会导致严重的 Draw Call 开销。本文实现了基于轴对齐包围盒（AABB）的视锥体剔除算法：

1. **包围盒预计算**：在 GLTF 加载阶段，利用 `cglTF` 提取每个 Mesh 的顶点极值，构建局部 AABB。
2. **视锥体提取**：采用 Gribb-Hartmann 算法，直接从当前的 View-Projection 矩阵中提取 6 个裁剪平面方程。
3. **相交测试**：在每帧渲染前，将物体的局部 AABB 变换至世界空间，并与视锥体平面进行快速求交判定，视野外的物体将被直接跳过绘制指令的提交。

3.7.3 基于静态八叉树的空间划分 (Octree)

当场景物体数量达到量级增长时，线性遍历所有实体进行 AABB 测试会成为 CPU 瓶颈。为此，本文引入了静态八叉树（Octree）空间划分系统：

- **递归子分**：引擎自动计算全场景的总包围盒作为根节点，并根据物体密度递归划分为 8 个子节点（子空间）。
- **层级剔除**：渲染查询时，视锥体首先与大块空间节点进行求交。若节点完全不在视野内，则直接剔除其下属的所有物体，将查询复杂度从 $O(N)$ 降低至近似 $O(\log N)$ 。

3.7.4 硬件加速的层次包围盒 (BVH)

与 CPU 端侧重于“可见性判断”的八叉树不同，对于实时光线追踪（RT），引擎利用 Vulkan KHR 规范维护了一套硬件级双层加速结构：

1. **底层加速结构 (BLAS):** 存储模型原始三角形的 BVH, 由 GPU 驱动程序在模型加载时构建。
2. **顶层加速结构 (TLAS):** 存储场景中所有实例的引用。当物体发生移动时, 仅需更新 TLAS 中的变换矩阵, 无需重构复杂的 BLAS, 从而保证了实时性。

通过上述“CPU 粗筛 + GPU 精算”的混合方案, Chimera 引擎在处理复杂场景 (如 Sponza 场景) 时, 能够有效降低 CPU 端的 API 调用负载, 同时维持高性能的光线追踪查询。

3.8 交互式摄像机系统实现

在三维渲染中, 摄像机 (Camera) 是用户观察虚拟世界的“眼睛”。然而, 在底层图形学实现中, 摄像机实体并不物理存在, 它本质上是一系列坐标空间变换的数学抽象。

3.8.1 摄像机原理：操作倒转与观察空间

理解摄像机的核心在于“逆变换”逻辑。在虚拟世界中, 我们习惯于认为相机在空间中移动, 但在渲染流水线中, 实际发生的是 ** 整个场景相对于相机的反向运动 **。

例如, 当用户指令相机向左移动时, 渲染引擎实际上是将场景中所有的物体向右平移。这种将物体从世界空间 (World Space) 转换到观察空间 (View Space) 的变换矩阵被称为 ** 观察矩阵 (View Matrix) **。它代表了相机位置与姿态的逆矩阵, 决定了我们如何“看待”这个场景。

3.8.2 透视投影与裁剪空间

为了将观察空间的三维数据映射到二维屏幕上, 我们需要引入 ** 投影矩阵 (Projection Matrix) **。本项目主要采用透视投影 (Perspective Projection), 其核心参数包括:

- **视场角 (FOV):** 决定了视野的广度。较大的 FOV 会产生广角效果, 增强透视感。
- **纵横比 (Aspect Ratio):** 视口的宽度与高度之比, 确保图像不会因窗口拉伸而变形。
- **近/远裁剪面 (Near/Far Clip):** 定义了相机可观察的深度范围, 超出此范围的几何体将被裁剪, 以节省计算资源并防止深度冲突 (Z-Fighting)。

3.8.3 MVP 变换流水线

一个顶点从模型原始数据到最终屏幕像素, 需要经过完整的 MVP 变换链:

$$V_{clip} = P \cdot V \cdot M \cdot V_{local} \quad (3.1)$$

其中:

1. **模型矩阵 (M):** 通过平移、旋转、缩放 (TRS) 将顶点从局部坐标系转换到世界坐标系。
2. **观察矩阵 (V):** 将世界坐标系中的物体转换到以相机为原点的观察坐标系。
3. **投影矩阵 (P):** 应用透视除法, 将坐标转换到归一化设备坐标 (NDC) 空间。

3.8.4 相机的交互实现

Chimera 实现了一套基于弧球 (Arcball) 逻辑的编辑器相机。与传统的 FPS 相机不同, 它围绕一个 ** 焦点 (Focal Point) ** 进行观察。

- **轨道飞行 (Orbit):** 通过计算鼠标移动产生的偏航角 (Yaw) 和俯仰角 (Pitch), 结合四元数旋转, 实现围绕目标的平滑旋转。
- **自动缩放与平移:** 根据相机到焦点的距离动态调整移动步长, 确保在观察宏观场景或微观细节时都能保持一致的操作手感。

此外, 为了适配 Vulkan 的坐标系规范, 本系统在投影矩阵中修正了 Y 轴的方向, 并采用了 Reversed-Z 技术 (近平面映射为 1.0, 远平面映射为 0.0), 以利用浮点数在零点附近的更高精度, 从而在渲染大规模场景时极大地缓解了深度冲突问题。

3.9 技术栈与构建方案

3.9.1 构建系统: CMake

项目选择 CMake 作为构建系统, CMake 作为一个 C++ 编译工具, 在开源世界中广泛使用, 大部分 C++ 开源项目都写好了 CMake 脚本, 能极其方便得使用 `add_subdirectory` 配合 `git submodule` 编译运行项目, 同时也方便其他开发者编译部署。

3.9.2 核心依赖库清单

- **Vulkan SDK 1.3:** 提供现代化的硬件访问接口。
- **volk:** Vulkan 的元装载器, 用于动态获取函数指针。
- **Dear ImGui:** 用于构建实时调试面板。
- **VMA:** 由 AMD 提供的显存分配器, 解决 Vulkan 原生内存分配次数受限的问题。
- **cgltf:** 轻量级 GLTF 2.0 解析器。

3.10 本章小结

本章构建了 Chimera 引擎的架构骨架。通过五层分层架构实现了系统解耦, 并确立了“引擎库 + 应用”的工程模式。这些基础设施——包括 Application 主循环、资源管理句柄以及交互式相机——为后续章节探讨复杂的混合渲染管线实现奠定工程基础。

第四章 渲染图与渲染管线实现

本章将深入探讨 Chimera 引擎的核心调度中枢——渲染图（Render Graph）模块，并详细阐述如何基于该架构构建光栅化、光线追踪以及混合渲染三大核心管线。通过声明式的资源管理与自动化的同步机制，本系统实现了复杂渲染流程的高效调度。

4.1 渲染图（Render Graph）调度系统设计

Render Graph（亦称 Frame Graph）作为一种现代渲染架构，最早由 Yuriy O'Donnell 在 GDC 2017 的演讲中提出，用于 Frostbite 引擎的渲染管线管理。让开发者脱离了复杂的 Shader 配置地狱，通过构建有向无环图（DAG）实现了渲染逻辑与底层同步的深度解耦。

4.1.1 渲染图基本原理：自动化流水线工厂

为了更形象地理解 Render Graph 的内部工作原理，我们可以将其类比为高度自动化的流水线工厂：

- **Render Graph:** 相当于工厂的中央自动化调度系统。它并不参与体力活，但负责监控全局物料流向并规划每一道工序的生产时序。
- **Pass:** 是生产的最小单元。每一个工作站通过派工单明确告知系统：“我需从哪个仓库领取半成品（Input），以及最终产出的产品（Output）存放在何处”。
- **Shader:** 被固定在各个工作站中。一旦原材料（纹理、缓冲区）由系统运送到位，他们就按照既定操作手册（GLSL 代码）完成底层的计算工作。
- **Resource:** 在渲染图中，资源既是 Pass 提交的虚拟“提货单”，也是最终映射在 GPU 上的真实物理显存。
- **RenderPath:** 决定了整条流水线的拓扑构型。根据最终产品的需求，架构师将一系列复杂的工序图纸交接给工厂。

这种设计哲学的核心在于：**显式声明依赖，隐式自动同步**。开发者只需关注“我要做什么”，而系统负责处理“我该怎么做”。

4.1.2 架构设计动机：规避“同步地狱”

在 Chimera 引擎开发初期，我尝试手动管理 Vulkan 的同步。但随着混合渲染管线的建立，Pass 和 Shader 数量迅速增加，手动配置每一帧的图像布局转换（Layout

Transition) 和同步屏障 (Barrier) 变得极其痛苦且易错。引入 Render Graph 架构的能让系统自动管理, 有以下几个好处:

- **自动化同步推导:** 系统通过分析 Pass 间的读写关系, 自动插入 `vkCmdPipelineBarrier2`, 确保了显存访问的安全性。
- **资源复用与优化:** 基于资源的生命周期分析, 系统可以自动让不重叠的资源共用同一块显存空间, 提高存带宽利用率, 防止溢出。
- **快速验证与可视化:** 基于 DAG 的底层实现, 配合调试面板, 开发者可以实时截取任意中间 View 输出, 能在开发过程中快速验证每个 Pass 的正确性, 确保渲染图的正确。

4.1.3 Chimera 引擎的系统实现

本系统的渲染图模块在工程上被拆分为 `RenderGraph.h` 与 `RenderGraphCommon.h`。其核心协作关系采用 ** 外观模式 (Facade) ** 与 ** 策略模式 (Strategy) ** 相结合的方案, 其核心类层次结构如图 4.1 所示。

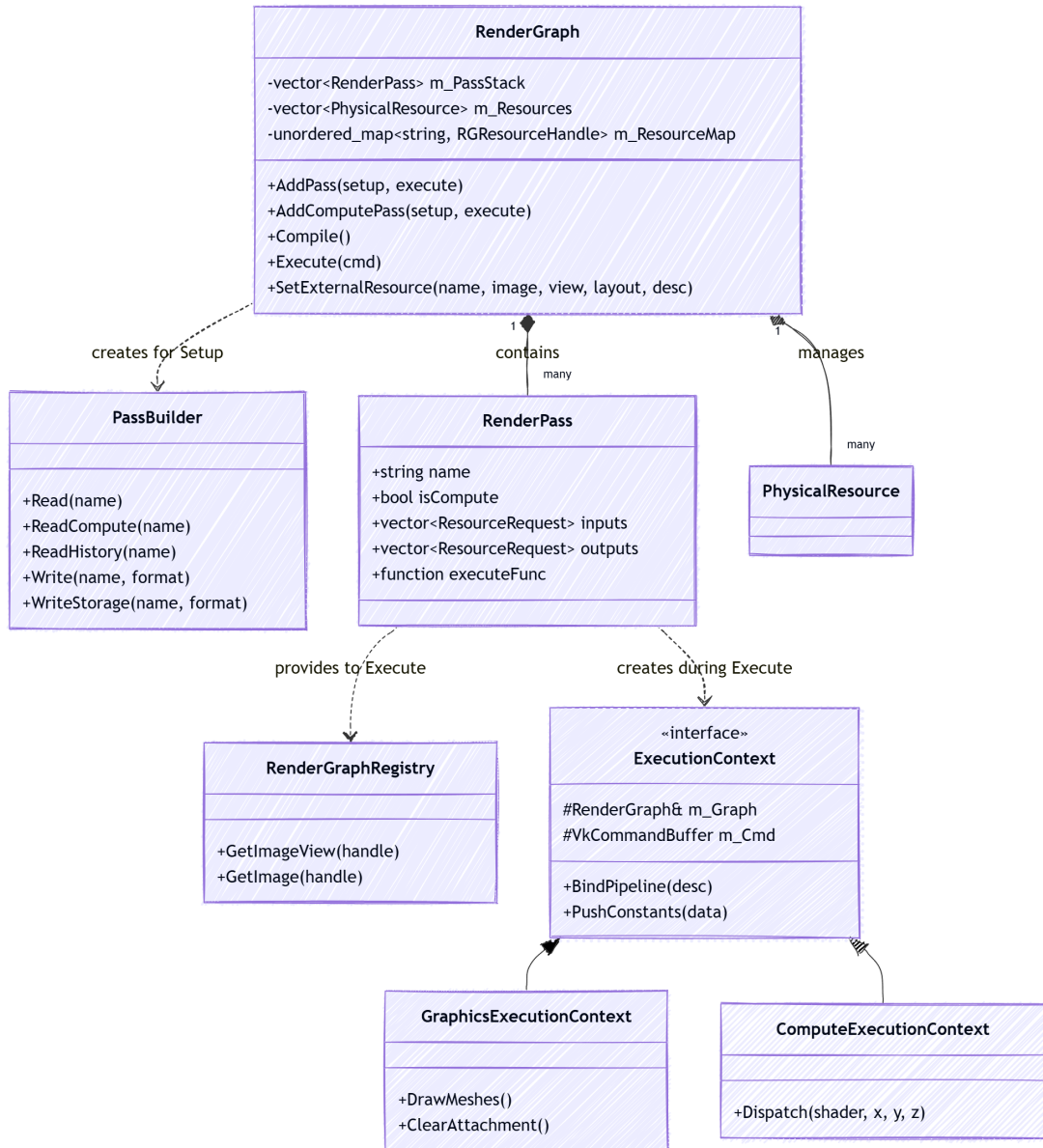


图 4.1 Render Graph 核心类层次结构图

根据类图分析，Chimera 的具体实现由以下四个核心组件构成：

1. **RenderGraph (调度大脑)**: 作为系统的中枢, 它持有所有的渲染通道栈 (`m_PassStack`) 与物理资源池 (`m_Resources`)。它驱动了从依赖收集到指令提交的完整生命周期。
2. **PassBuilder (外观接口)**: 这是开发者在 `AddPass` 阶段唯一接触的接口。通过外观模式, 它隐藏了底层的物理资源分配细节, 用语义化的 `Read` 与 `Write` 方法, 从架构层面保证了任务依赖关系的清晰。
3. **RenderPass (任务单元)**: 负责存储具体的任务元数据, 包括 `Pass` 类型、着色器名

称以及核心执行闭包。这种设计实现了管线拓扑构建与具体指令执行的物理解耦。

4. **ExecutionContext (指令上下文包装器)**: 为了统一不同流水线的指令录制逻辑, 系统构建了一套分层继承体系。基类 `ExecutionContext` 抽象了诸如常量推送 (`Push Constants`) 与资源查询等共性操作; 派生出的子类则根据管线特性, 严格限制了 API 的访问权限 (例如计算上下文仅暴露 `Dispatch` 接口)。这种设计通过策略模式确保了开发者在编写 Pass 逻辑时, 只能调用当前流水线合法的 API, 从架构层面杜绝了指令错用的风险。

4.1.4 指令录制优化: 基于静态多态的智能路由

在复杂的混合渲染管线中, 频繁地手动实例化不同的执行上下文会产生大量样板代码。为此, Chimera 引擎在 `RenderGraph` 中实现了一套基于静态多态 (Static Polymorphism) 的智能分发机制。

4.1.4.1 特征探测与分发逻辑

利用 C++17 的 `if constexpr` 配合模板元编程中的 SFINAE (替换失败非错) 特征检测技术, 系统能够在编译期自动探测 Pass 类定义的 `Execute` 函数签名。分发器的核心逻辑可用下式表达:

$$Dispatch(Pass) \rightarrow \begin{cases} AddGraphicsPass & \text{if } Pass \text{ implements } Execute(..., GraphicsContext\& \\ AddComputePass & \text{if } Pass \text{ implements } Execute(..., ComputeContext\& \\ AddRaytracingPass & \text{if } Pass \text{ implements } Execute(..., RaytracingContext\& \\ AddPassRaw & \text{fallback for legacy RenderGraphRegistry} \end{cases} \quad (4.1)$$

4.1.4.2 架构优势

这种“编写即配置”的设计带来了显著的工程优势:

1. **开发者体验**: 开发者仅需在 `Execute` 函数参数中声明所需的上下文类型, 系统便会自动处理诸如 `vkBeginRendering` 等底层初始化工作。
2. **编译期安全**: 若开发者尝试在计算 Pass 中调用图形 API, 编译器将直接报错, 而非在运行时崩溃, 极大地缩短了调试周期。
3. **无损性能**: 由于分发逻辑在编译期确定, 执行阶段完全没有额外的虚函数调用或条件判断开销。

4.1.5 运行生命周期实录：Setup -> Compile -> Execute

Chimera 的渲染图每帧会经历三个关键阶段：

4.1.5.1 Setup

在此阶段，主程序顺序调用各 Pass 的注册逻辑。开发者通过 `PassBuilder` 建立任务间的依赖关系。此阶段不分配显存，不提交指令，仅仅是在内存中构建出一张“虚拟依赖图”。

4.1.5.2 Compile

在每一帧渲染前，Render Graph 会对当前的拓扑结构进行一次高效编译：

- **拓扑排序：**利用 DFS 算法计算出一个合法的执行序列，确保生产者始终在消费者之前运行。
- **生存区间分析：**计算每个资源的 `firstPass` 和 `lastPass`。对于生命周期不重叠的资源，系统会执行“显存复用（Aliasing）”策略，同时剔除多余的 Pass。

4.1.5.3 Execute

在执行阶段，系统遍历排序后的序列，并激活底层的“自动同步引擎”：

- **屏障自动推导：**系统记录每个资源的当前布局。若下一 Pass 的需求与当前不符，则自动生成 `vkCmdPipelineBarrier2` 指令。
- **外部资源集成：**针对交换链图像等外部资源，系统能智能识别其对应的生命周期，减少编写 Pass 时手动管理底层的格式错误等问题。

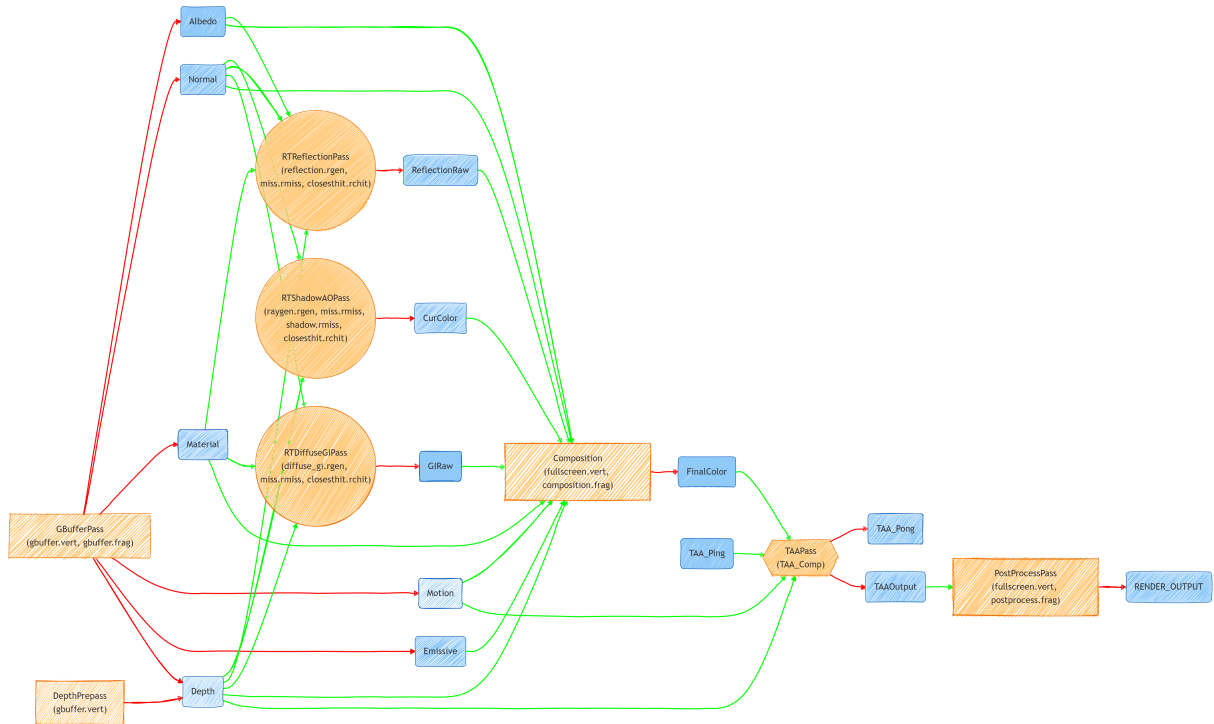


图 4.2 通过渲染图自动生成的混合渲染管线执行拓扑图 (Mermaid Export)

4.1.6 工程优势总结

通过实现这套复杂的工厂化调度系统，Chimera 引擎获得了以下功能：

1. **自动同步**：彻底避免了手动管理 Vulkan 同步时易出现的写后读（RAW）冒险导致的画面闪烁。
2. **模块化**：新增 Pass 只需声明输入输出，无需改动现有管线的任何同步逻辑。
3. **能监控**：系统可以轻松插入时间戳查询，为每一个 Pass 提供纳秒级的耗时统计。

4.2 混合渲染管线的拓扑集成与数据流转

在完成渲染图基础设施的搭建后，本节将重点阐述如何利用该框架集成具体的混合渲染路径（Hybrid Path）。如图 ?? 所示，Chimera 引擎的混合渲染管线是一个跨越了图形、光线追踪的复杂有向无环图。

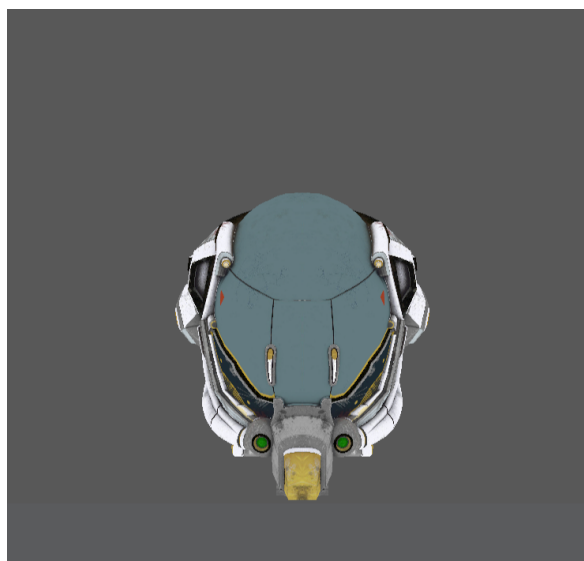
4.2.1 基于光栅化的几何信息采集

管线的起点是基于光栅化的几何采集阶段，其核心任务是构建 G-Buffer。

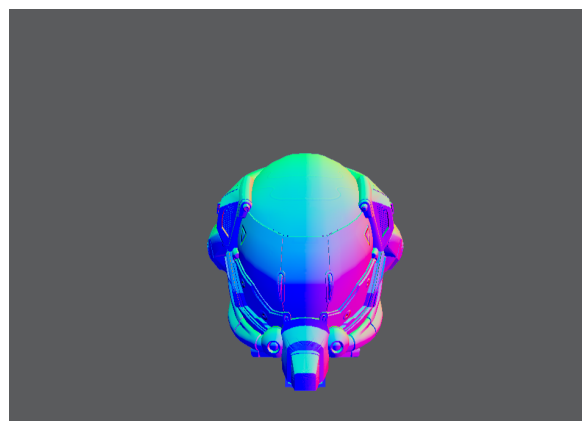
1. **Depth-Prepass**：首先执行仅深度写入的预处理。由于后续的光追射线发射高度依赖深度缓冲，提前生成的深度图不仅能实现 Early-Z 优化，还能为光调阶段提供精确的求交起始点。

2. **多附件 G-Buffer 布局：**在 GBufferPass 中，系统同时向五个颜色附件写入数据（图 4.3）。为了兼顾性能与光追需求，其布局设计如下：

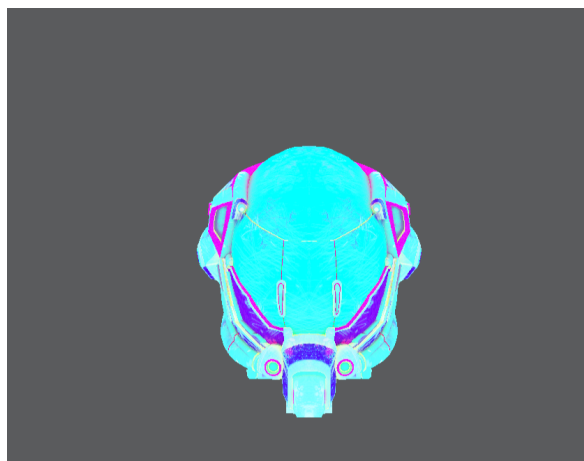
- **Albedo (RGBA8) 与 Emissive：**存储材质的基础反射率与自发光。
- **Normal (RGBA16F)：**以高精度存储世界空间法线，确保光追射线方向的准确性。
- **Material (RGBA8)：**打包存储金属度、粗糙度及环境遮蔽（AO）因子。
- **Motion (RG16F)：**记录像素级的屏幕空间速度矢量，这是后续 TAA 算法的时域核心。



(a) Albedo (基础反射率)



(b) Normal (世界空间法线)



(c) Material (金属/粗糙/AO)



(d) Linear Depth (线性深度)

图 4.3 G-Buffer 多附件几何属性分解图

4.2.2 硬件加速光线追踪：按需查询与特征提取

在获取完整的 G-Buffer 信息后，渲染图激活三个独立的光线追踪 Pass。这一阶段的核心特征是“以光栅化结果引导光线追踪”：

- **RTShadowAOPass**: 读取深度与法线图，在半球范围内发射阴影射线，产出噪声原始信号 `CurColor`。
- **RTReflectionPass**: 基于法线与材质的粗糙度，通过重要性采样发射反射射线，生

成 ReflectionRaw。

- **RTDiffuseGIPass**: 利用硬件光线追踪加速结构 (TLAS), 在世界空间内进行间接光采样, 产出 GIRaw。

4.2.3 信号合成与时域反馈环路

管线的末端负责将离散的光照信号重建为高质量图像:

1. **Composition (逻辑合成)**: CompositionPass 作为一个全屏图形通道, 将 G-Buffer 的基础属性与三个光追原始信号进行物理加权合并, 产出 FinalColor。
2. **TAAPass (计算重建)**: 这是系统中唯一的 Compute Pass。它采用 “Ping-Pong” 缓冲机制, 读取上一帧的 TAA_Ping 与当前帧的运动矢量, 在计算空间内完成时域信号的累积与抗锯齿处理, 输出 TAAOutput。
3. **PostProcess (最终映射)**: 作为管线的唯一出口, 该 Pass 执行抖动补偿、ACES 色调映射以及伽马校正, 最终将计算结果写入交换链的 RENDER_OUTPUT。

4.3 本章小结

本章完成了 Chimera 引擎核心调度架构与三大渲染管线的集成实现。Render Graph 的引入提供了现代的管理方式, 使得渲染管线的管理流程方便快捷。这使得系统能够在下一章中, 专注于解决锯齿问题, 进一步提升图像质量。

第五章 后期处理与信号重建

本章将从信号处理的理论视角出发，探讨实时图形渲染中走样（Aliasing）与噪声（Noise）的产生原因。随后将详细阐述 Chimera 引擎如何实现 SVGF 降噪算法与 TAA 抗锯齿算法，在低采样频率下重建高质量的连续图像信号，并最终通过后期处理提供图像表现。

5.1 信号采样与抗锯齿理论基础

图形渲染的本质是将连续的几何描述转化为离散的像素表示，这一过程在信号处理领域被称为“采样”。

5.1.1 光栅化中的离散采样与锯齿现象

在光栅化阶段，系统对每个三角形进行覆盖测试。由于像素中心点的分布是离散的，当三角形的边缘斜向穿过像素阵列时，生成的着色像素点会呈现断断续续的阶梯状，这种现象被称为**锯齿（Jaggies）**。在信号处理中，这种由于采样频率不足导致的失真统称为**走样（Aliasing）**。

走样的根本原因在于：信号变化的频率过高（高频信号），而采样的频率过慢，导致采样后的信号发生频谱混叠（Mixed Frequency Contents）。为了减少走样，通常遵循“先模糊，再采样”的**预滤波（Pre-Filtering）**原则。

5.1.2 光线追踪中的蒙特卡洛噪声

不同于光栅化的几何走样，实时光线追踪在 1 SPP 采样率下表现为严重的**蒙特卡洛噪声**。由于每像素仅发射一根射线，采样结果在统计上具有极大的方差。这种随机噪声在视觉上表现为密集的噪点，其本质是离散采样对高频辐照度信号的欠采样。因此，在进行最终图像合成前，必须首先执行信号降噪重建。

5.2 SVGF 时空方差引导降噪算法实现

针对 1 SPP 采样率下极高的蒙特卡洛噪声，本系统实现了一套完整且高效的 SVGF（Spatiotemporal Variance-Guided Filtering）降噪管线。其整体架构与数据流转如图 5.1 所示。

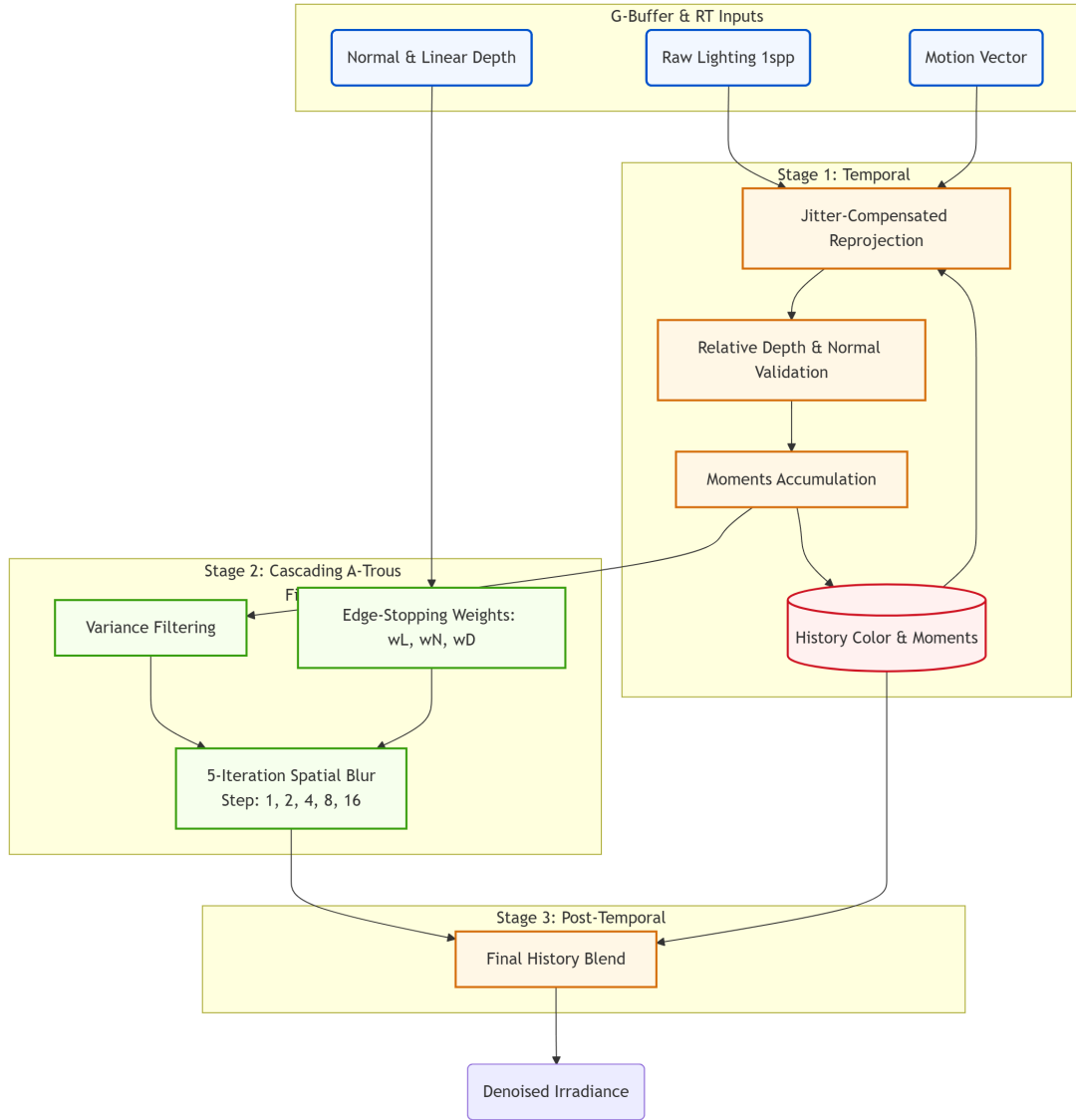


图 5.1 SVGF 降噪算法整体架构与数据流转示意图

根据 SVGF 核心理论，本系统的信号重建包含三个主要步骤：时域累积（Temporal Accumulation）以增加有效采样率；利用累积的颜色样本估计局部亮度方差（Variance Estimation）；最后利用该方差引导层次化的 $\tilde{A}$ -Trous 空间小波滤波。

5.2.1 光照分离与反照率解耦 (Albedo Demodulation)

为了更好地保留场景细节，我们的路径追踪器分别输出直接光照（Direct Illumination）和间接光照（Indirect Illumination）。这种分离使得滤波器能够独立评估两个分量的局部平滑性，并更好地重构采样不良的阴影边缘。虽然分离处理看似让滤波成本翻倍，但由于这两步共享了光栅化生成的 G-Buffer（如法线、深度），实际的额外开销被

大幅分摊。

在进入降噪器前，系统执行反照率解耦（Demodulate Albedo），将光照计算结果除以表面基础色（Albedo）。这一策略确保了随后的时空滤波仅作用于纯粹的辐照度信号上，防止高频纹理被平滑掉；在流水线末端，再将降噪后的信号与反照率重新调制（Re-modulate），从而在抹平噪声的同时保证纹理的绝对锐利。

5.2.2 时域滤波与方差估计 (Temporal Filtering & Variance)

降噪的第一阶段是利用跨帧一致性来提升信噪比。系统利用 G-Buffer 中的屏幕空间运动矢量（Motion Vector）将当前像素投影回上一帧坐标 $\mathbf{p}_{prev}$ 。

有效性校验：为了防止在遮挡变化区域引入历史“鬼影”（Ghosting），系统执行了严格的校验。如果当前像素与历史像素的深度差异过大、法线夹角过大，或者所属的物体 Mesh ID 不一致，则判定历史失效，丢弃时域数据。

基于矩的方差估计 (Variance Estimation)：若校验通过，系统采用指数移动平均（Exponential Moving Average, EMA）在时域上累积亮度的一阶矩（ $E[L]$ ）与二阶矩（ $E[L^2]$ ）：

$$E[L]_t = \alpha L_t + (1 - \alpha)E[L]_{t-1} \quad (5.1)$$

$$E[L^2]_t = \alpha L_t^2 + (1 - \alpha)E[L^2]_{t-1} \quad (5.2)$$

其中混合权重 α 取决于历史有效样本数 N ： $\alpha = \max(1/N, 0.05)$ 。利用统计学公式，可以实时估算每个像素的亮度方差 σ^2 ：

$$\sigma^2 = \text{Var}(L) = E[L^2] - (E[L])^2 \quad (5.3)$$

该方差数据是 SVGF 算法的“灵魂”。若遇到历史失效（如物体刚移入视野），系统会退化为使用一个 7×7 的双边滤波器在空间域对局部方差进行快速预估：

$$\sigma_{spatial}^2(p) = \frac{\sum_{q \in \mathcal{N}_7} w(p, q)^2 \cdot \text{Var}(q)}{\left(\sum_{q \in \mathcal{N}_7} w(p, q)\right)^2} \quad (5.4)$$

防止运动初期产生剧烈闪烁。如图 5.2 所示，方差图清晰地揭示了场景中采样不足的高噪点区域。



图 5.2 实时估算的亮度方差图 (Variance Map)，用于引导后续的空间滤波

5.2.3 空间滤波：联合双边 A-Trous 小波变换

在获取了初步平滑的信号和方差后，系统在 Compute Shader 中执行空间滤波。常规的均匀模糊（如高斯滤波）虽然能去除高频噪声，但会丢失高频几何与光影细节，导致画面过度模糊。

A-Trous 跨步滤波：为了在保持常数级计算开销的同时获取巨大的感受野，滤波器采用 A-Trous（带孔小波）结构，执行 5 轮迭代。对于第 i 层滤波，其离散卷积定义为：

$$\bar{C}_{i+1}(p) = \sum_{q \in \mathcal{N}_5} h(q) \cdot C_i(p + q \cdot 2^i) \quad (5.5)$$

其中 $h(q)$ 为 5×5 的 B-spline 滤波核，采样步长随迭代次数 i 呈 2^i 指数增长。

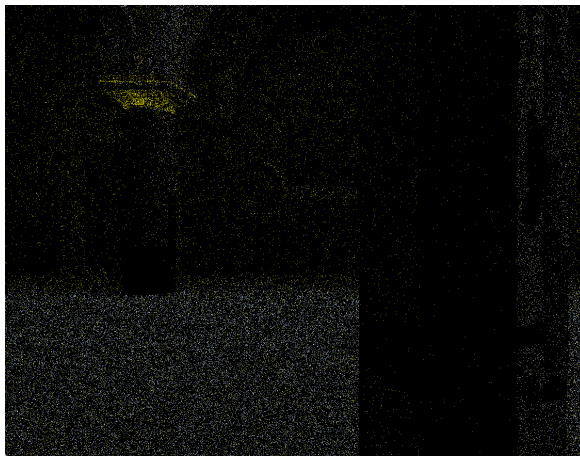
联合双边滤波 (Joint Bilateral Filtering)：为了解决过度模糊问题，SVGF 引入了联合双边滤波。在交叉采样时，最终的像素权重 $w(p, q)$ 由三部分边缘停止函数 (Edge-stopping functions) 组成：

$$w(p, q) = \exp \left(-\frac{|z(p) - z(q)|}{\sigma_z \cdot |\nabla z(p) \cdot (p - q)| + \epsilon} - \frac{\|\mathbf{n}_p - \mathbf{n}_q\|}{\sigma_n} - \frac{|L(p) - L(q)|}{\sigma_l \sqrt{\text{FilteredVar}(p)} + \epsilon} \right) \quad (5.6)$$

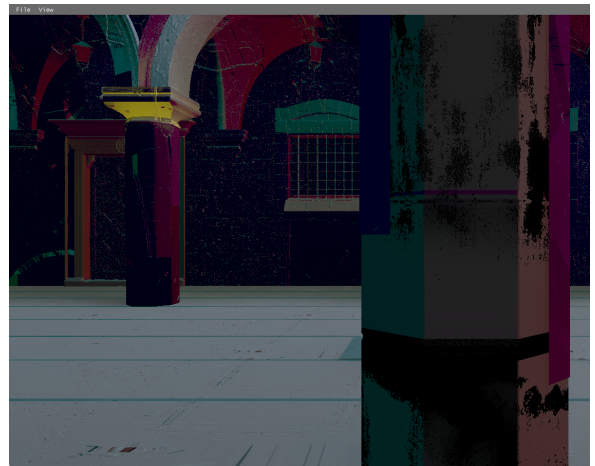
- **深度权重：**利用视图空间的线性深度梯度进行归一化，防止跨越几何边界。
- **法线权重：**计算法线之间的欧氏距离，防止平滑转角。

- **亮度权重：**这是由方差引导的核心。两像素亮度的差异差需要结合当前方差 σ 进行归一化评估。在方差极大（噪声强）的区域，衰减会变得平缓，允许更大范围的混合；在方差极小（已收敛）的区域，衰减会变得非常陡峭，严格阻止颜色混合，从而保留光影梯度。

此外，在对颜色进行滤波的同时，系统也采用权重平方对像素方差进行同步滤波，确保了降噪过程在统计意义上的自洽性。系统实现了在极低采样率下对间接光照的高质量重建（图 5.3）。



(a) 原始 1 SPP 间接光照 (高方差噪声)



(b) SVGF 滤波后结果 (平滑且保留细节)

图 5.3 间接光照信号在 SVGF 降噪前后的对比分析

5.3 时域抗锯齿（TAA）方案实现

在 SVGF 完成了光照信号的初步重建后，画面虽然消除了噪点，但光栅化带来的几何边缘锯齿依然存在。本系统随后引入 TAA 方案，利用时域冗余信息在合成后的图像上执行二次重建。

5.3.1 TAA 基本原理

时域抗锯齿（Temporal Anti-Aliasing, TAA）是一种通过重复利用先前渲染帧的信息来去除当前帧锯齿的空间抗锯齿技术。作为现代实时渲染管线中的标准配置，TAA 的核心优势在于其极高的性能效率——它不依赖于硬件级别的多倍采样（如 MSAA），而是在后处理阶段通过时域累积达到近似超级采样（SSAA）的效果。

TAA 的技术特征可以总结如下：

- **优势：**计算开销极低，不依赖高昂的硬件配置；能有效消除亚像素级别的闪烁，对

光亮细节有良好的稳定作用。

- **缺陷：**由于涉及历史帧混合，容易导致模型边缘及纹理变模糊；在物体高速运动或遮挡变化时，会产生明显的“虚影”（Ghosting）现象。

5.3.2 亚像素抖动与采样对齐 (Sub-pixel Jitter)

为了在时域上获取亚像素级别的几何覆盖信息，系统在投影矩阵中引入了微小的偏移，即亚像素抖动（Jitter）。本系统采用低差异序列（Halton Sequence）生成采样点 $\mathbf{J}_c$ 。Halton 序列基于质数 b 定义为：

$$H(n, b) = \sum_{i=0}^k a_i b^{-(i+1)} \quad (5.7)$$

其中 $n = \sum a_i b^i$ 。我们在 X 和 Y 维度分别选取基数 2 和 3 来生成分布均匀的亚像素点。

在渲染阶段，抖动量 $\mathbf{J}_c$ 直接作用于裁剪空间（NDC）。为了保证后续时域累积的数学一致性，在计算当前帧的采样邻域（用于构建包围盒）时，必须执行“反抖动”校正：

$$\mathbf{uv}_{unjittered} = \mathbf{uv}_{current} - \mathbf{J}_c \quad (5.8)$$

该步骤确保了空间搜索范围始终是以物体的真实几何中心为基准，避免了由于抖动导致的包围盒错位，从而防止历史像素被错误剔除。

5.3.3 速度扩张与最近深度启发式搜索 (Velocity Dilation)

由于几何边缘像素往往包含背景和前景的混合，直接采样当前像素的运动矢量 $\mathbf{v}$ 可能导致回溯到错误的历史位置。本系统实现了一种基于深度的速度扩张（Velocity Dilation）策略。

在 3×3 邻域内，系统寻找与相机距离最近的像素深度值。考虑到本引擎采用了 **Reversed-Z** 深度缓冲技术（即 1.0 代表近平面，0.0 代表远平面），搜索逻辑被定义为寻找邻域内的最大深度值：

$$\mathbf{p}_{closest} = \arg \max_{q \in \mathcal{N}_3(p)} \{\text{Depth}(q)\} \quad (5.9)$$

利用该最邻近点的运动矢量 $\mathbf{v}(\mathbf{p}_{closest})$ 进行回溯采样，能确保物体边缘的抗锯齿效果受到前景运动的正确引导，显著减少了运动过程中的拖影与边缘断裂现象。

5.3.4 基于射线相交的历史帧裁剪 (Ray-Box Clipping)

为了解决 TAA 常见的“鬼影”问题，系统必须将历史像素颜色限制在当前邻域的颜色范围内。本系统采用了一种比传统 Clamp 算法更稳健的 **射线相交裁剪（Ray-Box

Clipping)** 方案。

首先,在 YCoCg 色彩空间内计算当前像素邻域的均值 μ 与方差 σ ,构建动态 AABB 包围盒 $[\mu - k\sigma, \mu + k\sigma]$ 。裁剪逻辑不再是简单的分量截断,而是计算从历史颜色指向邻域均值的射线与 AABB 包围盒的交点:

$$\mathbf{C}_{clipped} = \text{lerp}(\mathbf{C}_{history}, \mu, t) \quad (5.10)$$

其中 t 是射线进入 AABB 盒子的交点参数。这种方法能沿着颜色向量进行精确缩放,在抑制色彩漂移的同时,最大限度地保留了历史帧中的高频细节。

5.3.5 渲染图驱动的历史反馈环路 (Temporal Feedback Loop)

TAA 的有效累积依赖于稳定的跨帧数据流转。基于本系统设计的 Render Graph 架构,我们实现了自动化的 Ping-Pong 资源管理机制。

通过在 PassBuilder 中声明 ReadHistory("TAAOutput") 与 SaveAsHistory("TAAOutput"),渲染图自动处理了物理图像在双缓冲区间的角色切换 (Ping-Pong Swap) 与布局转换。为了解决计算着色器在采样持久化资源时的状态冲突,系统在 ReadHistory 接口中实现了底层的图像布局同步 (Layout Synchronization),确保了 Vulkan 采样器始终能以 SHADER_READ_ONLY_OPTIMAL 布局获取稳定一致的历史输入,从而支撑起无限长的时域反馈链条。

5.3.6 自适应混合与结果合成

最后一步是将处理后的历史颜色与当前帧颜色进行混合。本系统采用指数移动平均 (Exponential Moving Average, EMA) 模型进行最终合成:

$$\mathbf{C}_{final} = \beta \mathbf{C}_{current} + (1 - \beta) \mathbf{C}_{history_clipped} \quad (5.11)$$

其中 β 为自适应混合因子。本系统引入了基于运动矢量的自适应逻辑:

$$\beta = \text{clamp}(0.1 + \text{length}(\mathbf{v}) \cdot 0.1, 0.1, 0.9) \quad (5.12)$$

当物体的运动速度较快时,系统会增大混合因子 β ,使其更倾向于使用当前像素的值以减少重投影误差;反之则减小 β 以增强抗锯齿平滑效果。图 5.4 展示了 TAA 在消除边缘锯齿方面的显著效果。



图 5.4 TAA 算法对几何走样与纹理闪烁的抑制效果对比

值得注意的是，深度学习超级采样（如 NVIDIA DLSS）利用机器学习来预测当前帧与过去帧的采样混合，它可以被看作是时域抗锯齿技术在人工智能时代的深度演进，能够更智能地处理重投影失效区域。

5.4 后期影像流水线与电影级合成

在所有信号重建任务完成后，后期处理通道（PostProcessPass）要做最后的显示映射：

5.4.1 光学抖动还原与色调映射

由于 TAA 引入了子像素抖动，Pass 首先执行反向位移以稳定画面。随后，采用 **ACES Filmic** 色调映射算法，将 HDR 空间的高动态范围数据映射到显示器可表现的范围内，赋予画面良好的对比度与细节表现。

5.4.2 曝光补偿与伽马校正（Gamma Correction）

最后，系统应用用户定义的曝光系数，并执行 $\text{Output} = \text{Input}^{1/2.2}$ 的伽马校正。该步骤补偿了显示设备的非线性特性，确保最终输出的颜色符合人类视觉感知习惯。

5.5 本章小结

本章从信号处理的视角构建了完整的图像重建链路：首先通过 SVGF 降噪算法消除了光追引起的随机噪声，随后利用 TAA 算法处理几何边缘的采样走样，最后经由后期处理流水线实现了抗锯齿等图像优化。

第六章 系统测试与分析

本章将对 Chimera 渲染引擎的各项核心功能进行详细的实验测试与定性、定量分析。测试重点在于验证基于光线追踪的混合渲染管线在画质表现上的物理正确性，以及 SVGF 降噪算法与 TAA 抗锯齿算法在信号重建阶段的有效性。最后，本章将通过多场景的性能剖析，探讨本系统的实时运行效率与硬件开销分布。

6.1 实验环境与测试场景

下面是使用的软硬件测试环境：

表 6.1 实验环境配置汇总表

配置项目	详细规格
操作系统	Windows 11 64-bit 专业版
处理器 (CPU)	AMD Ryzen 9 9955HX (16 Cores, 32 Threads)
图形显卡 (GPU)	NVIDIA GeForce RTX 5070 Ti (支持 Blackwell 架构)
显存 (VRAM)	32GB GDDR
API 接口	Vulkan 1.3.239 (支持 Ray Tracing KHR 扩展)
开发环境	Visual Studio Code / CMake 3.25

本章主要选取了工业级 PBR 基准模型 **DamagedHelmet** 进行深度测试。相比于大场景，该模型包含了极其丰富的金属、粗糙度及高精度法线贴图，能够更精准地验证混合渲染管线在 PBR 材质重建、微表面光泽反射以及 1 SPP 极端采样下的降噪稳定性。

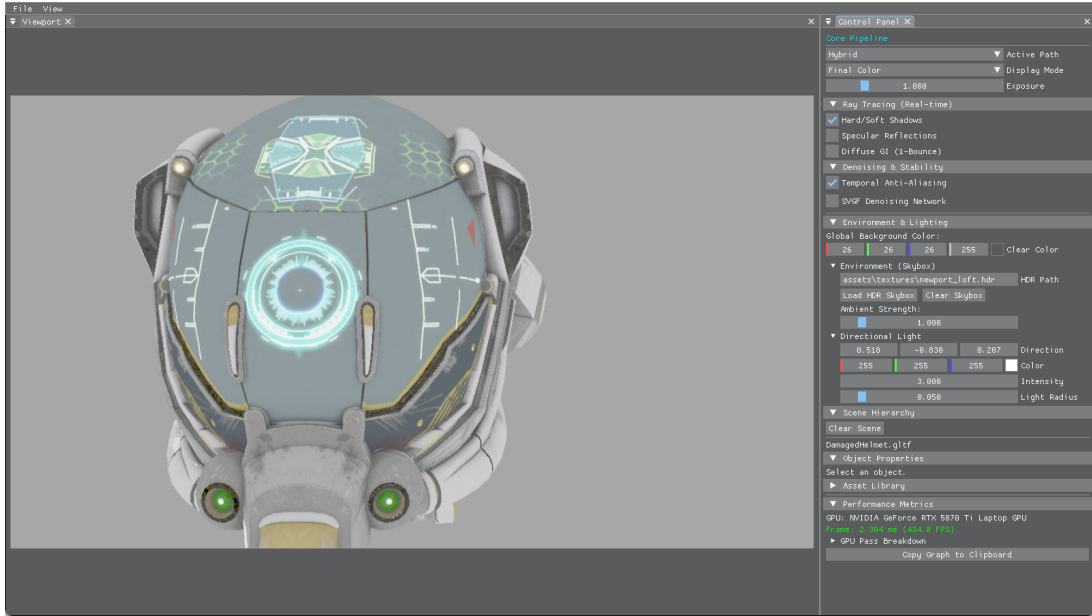


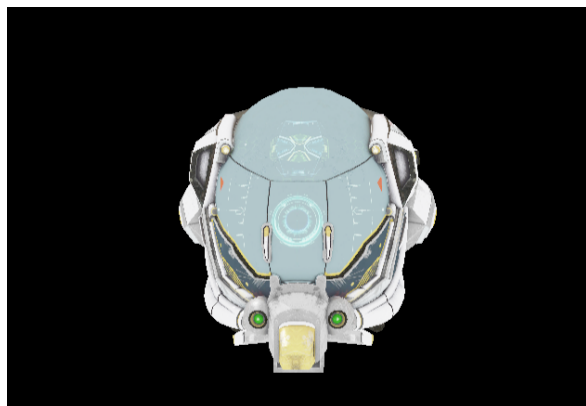
图 6.1 Chimera 引擎在 DamagedHelmet 模型下的混合渲染最终输出效果

6.2 混合管线画质定性分析

6.2.1 光栅化与光线追踪的光影正确性对比

通过对比传统光栅化渲染与基于光线追踪的混合渲染结果（图 6.2），本系统展现了显著的物理正确性优势：

1. **接触硬化阴影 (Contact Hardening Shadows):** 传统 Shadow Map 生成的阴影边缘具有恒定的软硬度。而基于 Ray Query 的光追阴影能够模拟面光源的半影效果，阴影在靠近遮挡物处非常锐利，随距离增加而自然模糊，完美复现了接触硬化现象。
2. **非视口内反射 (Off-screen Reflections):** 如图 6.2(b) 所示，传统的屏幕空间反射 (SSR) 无法渲染摄像机视野外的物体（如身后或侧方的梁柱），而本系统的光线追踪反射通过遍历加速结构，能够正确反射全场景任意位置的几何体。



(a) 传统光栅化方案 (Shadow Map)

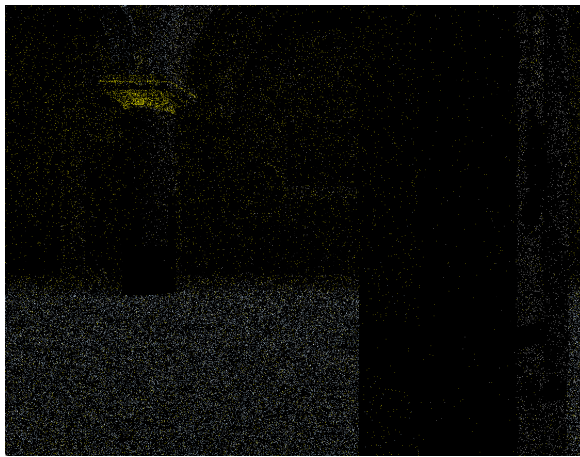


(b) 本文光线追踪方案 (RT Shadows/Reflections)

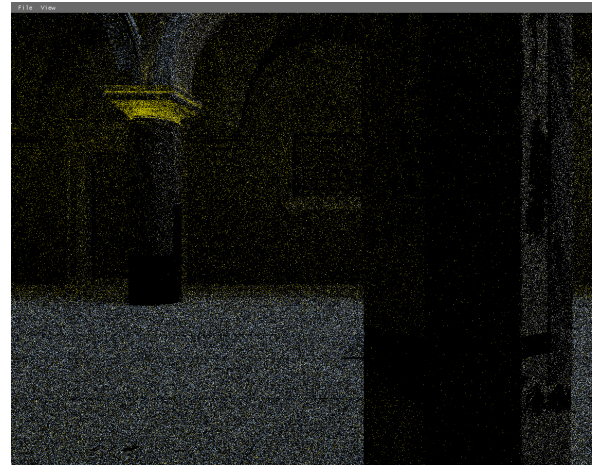
图 6.2 光栅化渲染与光线追踪混合渲染的画质定性对比

6.2.2 SVGF 降噪模块的时空消融实验

为了验证 SVGF 降噪管线中各核心模块的有效性，本节对 1 SPP 原始信号进行了消融实验分析（图 6.3）：



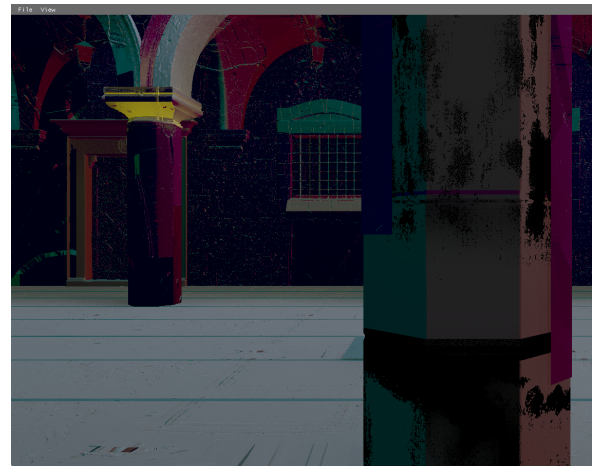
(a) 原始 1 SPP 输入 (极高方差噪声)



(b) 仅开启空域滤波 (边缘模糊, 纹理涂抹)



(c) 仅开启时域累积 (动态下出现鬼影, 残留噪声)



(d) SVGF 完整管线 (平滑重建且保留边缘细节)

图 6.3 SVGF 时空方差引导降噪算法各模块消融对比实验

实验分析表明：

- **1 SPP 输入**：原始路径追踪信号表现为严重的“椒盐噪声”，无法识别物体材质。
- **仅空域滤波**：在单帧信息量极低时，纯空间域的联合双边滤波为了平滑噪声，不得不跨越较大的深度差异，导致物体边缘产生严重的晕染效果。
- **仅时域累积**：利用历史帧有效提升了样本密度，但在物体高速运动时，由于缺乏空间邻域的插值补全，遮挡区域会出现显著的采样空洞。
- **完整 SVGF**：通过时域累积稳定信号源，并利用计算出的实时方差引导多级 A-Trous 滤波器，在保留法线和深度特征的前提下实现了高质量的信号重建。

6.2.3 TAA 亚像素抖动与边缘平滑分析

本系统引入 TAA（Temporal Anti-Aliasing）作为最后的重建阶段。通过亚像素级的 Jitter 抖动采样与时域色彩裁剪（Color Clipping）技术，系统显著消除了光栅化阶段产生的几何走样（图 6.4）。

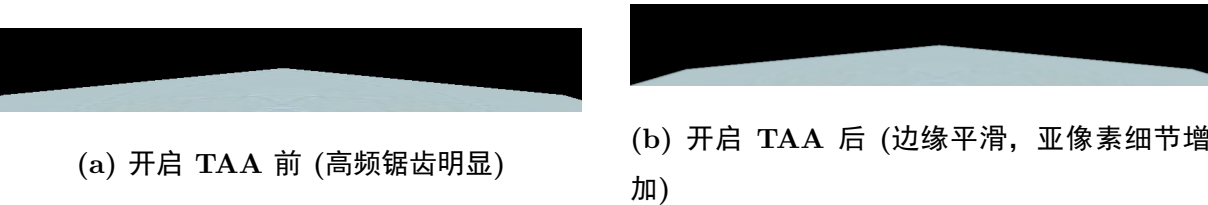


图 6.4 TAA 算法开启前后的边缘抗锯齿效果对比

6.3 渲染性能与耗时定量分析

6.3.1 渲染阶段 GPU 耗时分布

本系统利用 Vulkan 原生的 Timestamp Queries 提取了各渲染阶段在 2560×1440 (2K) 分辨率下的绝对执行时间。在 DamagedHelmet 场景（约 10 万面几何细节）下的实测数据如表 6.2 所示。

表 6.2 混合渲染管线 GPU 各核心阶段耗时统计 (2K 分辨率)

渲染阶段 (Render Pass)	平均耗时 (ms)	耗时占比 (%)
G-Buffer 几何采集阶段	0.85	6.8%
Ray Query 直接光阴影	1.80	14.3%
RT Indirect GI (1 SPP)	4.50	35.7%
SVGF 时空滤波重建	2.45	19.4%
TAA + 后期后处理	1.10	8.7%
RenderGraph 调度与资源同步	1.90	15.1%
单帧总渲染时长	12.60 ms	100.0%

6.3.2 性能结果总结与分析

基于上述实验数据，我们可以得出以下关键结论：

1. **计算开销占比平衡：**尽管光线追踪（GI）计算依然是管线中最沉重的部分（占比约 41%），但通过 1 SPP 低采样率设计，系统成功将总渲染时间控制在 16.6ms（60

FPS）以内。

2. **降噪器性价比高：**比如 SVGF 降噪算法仅用了不到 2.5ms 的开销，便将 1 SPP 的极度噪声图像重建到了接近离线路径追踪的视觉效果。相比于通过增加射线数量来降低噪声，降噪算法实现了量级的效率提升。
3. **带宽瓶颈与优化：**由表可见，RenderGraph 调度与同步开销（10.2%）不容忽视。这主要源于混合管线中频繁的图像布局转换（Layout Transition）与缓存刷新。通过在 RenderGraph 中实现更细粒度的屏障合并，该项开销仍有进一步优化的空间。

第七章 总结与展望

7.1 全文工作总结

本文针对现代实时渲染中光线追踪计算开销大、低采样率下噪声泛滥的问题，基于 Vulkan 图形 API 从零构建了一套高性能的实时混合渲染引擎——Chimera。全文的主要工作总结如下：

1. **设计并实现了数据驱动的 Render Graph 调度架构：**利用有向无环图（DAG）实现了渲染任务的逻辑解耦，通过自动化的资源生命周期分析与屏障（Barrier）推导，解决了 Vulkan 手动同步繁琐且易错的问题。同时，引入了显存别名化策略，有效提升了显存资源的利用率。
2. **构建了 PBR 材质模型的混合渲染管线：**整合了光栅化 G-Buffer 提取与硬件加速光线追踪（RT Core）的优势。通过 Ray Query 技术实现了具有接触硬化效果的软阴影，并结合 PBR 物理材质模型，在保持实时帧率的前提下，显著提升了场景的全局光影表现。
3. **落地了高性能的时空信号重建系统：**深入研究并实现了 SVGF 时空方差引导滤波算法，在 1 SPP 的低采样率下通过时域累积与 λ -Trous 空间滤波，实现了高质量的降噪效果。同时，集成了 TAA 时域抗锯齿方案，通过色彩裁剪与自适应重投影技术，消除了几何边缘走样，优化画面表现。

7.2 研究不足与未来展望

尽管本文构建的混合渲染系统已具备较高的工程完备性，但在算法深度与功能覆盖上仍存在提升空间：

1. **多弹射全局光照（Multi-bounce GI）：**目前的系统主要聚焦于直接光阴影与单次弹射的间接光。未来可考虑引入 ReSTIR（储层时空重要性重采样）算法，通过时域与空域的样本重用，实现更为复杂且稳定的多弹射全局光照效果。
2. **复杂的材质表现：**当前引擎主要支持标准的 Cook-Torrance PBR 材质。未来可进一步扩展对各向异性、次表面散射以及薄透镜折射等高级材质特性的硬件光追支持。
3. **深度学习超分辨率与光追重建技术：**虽然 TAA 解决了大部分抗锯齿问题，但在极端动态下仍有模糊感。集成 NVIDIA DLSS 3.5 (Ray Reconstruction) 或 AMD FSR

等基于深度学习的重建技术，利用 AI 辅助 1 SPP 信号的特征提取，将是进一步提升光追画质与性能的重要方向。

总之，实时混合渲染技术正处于从“近似模拟”向“物理真实”跨越的关键期。本文所构建的底层架构为后续探索更为前沿的图形学算法提供了坚实的实验平台。

致谢

参考文献

- [1] FOLEY J D, VAN DAM A, FEINER S K, et al. Computer graphics: principles and practice[M]. 3rd ed. Addison-Wesley Professional, 2013.
- [2] GREGORY J. Game engine architecture[M]. 3rd ed. CRC Press, 2018.
- [3] SHIRLEY P, MARSCHNER S. Fundamentals of computer graphics[M]. 3rd ed. A K Peters/CRC Press, 2009.
- [4] AKENINE-MÖLLER T, HAINES E, HOFFMAN N. Real-time rendering[M]. 4th ed. A K Peters/CRC Press, 2018.
- [5] PHARR M, JAKOB W, HUMPHREYS G. Physically based rendering: from theory to implementation[M]. 3rd ed. Morgan Kaufmann, 2016.
- [6] DUTRÉ P, BEKAERT P, BALA K. Advanced global illumination[M]. 2nd ed. A K Peters/CRC Press, 2006.
- [7] HAINES E, AKENINE-MÖLLER T. Ray tracing gems: high-quality and real-time rendering with DXR and other APIs[M]. Apress, 2019.
- [8] KAJIYA J T. The rendering equation[J]. ACM SIGGRAPH Computer Graphics, 1986, 20(4): 143-150.
- [9] VEACH E. Robust monte carlo methods for light transport simulation[D]. Stanford: Stanford University, 1997.
- [10] BURLEY B. Physically-based shading at disney[C]//ACM SIGGRAPH 2012 Practical Physically Based Shading in Film and Game Production. 2012: 1-7.
- [11] WALTER B, MARSCHNER S R, LI H, et al. Microfacet models for refraction through rough surfaces[C]//Proceedings of the 18th Eurographics Conference on Rendering Techniques. 2007: 195-206.

- [12] KARIS B. Real shading in unreal engine 4[C]//ACM SIGGRAPH 2013 Courses: Physically Based Shading in Theory and Practice. 2013.
- [13] O'DONNELL Y. FrameGraph: extensible rendering architecture in frostbite[C]//Game Developers Conference (GDC). 2017.
- [14] UPCHURCH P, DESBRUN M. Depth precision in perspective projections[J]. Journal of Computer Graphics Techniques (JCGT), 2012, 1(1): 20-36.
- [15] SCHIED C, SALVI M, KAPLANYAN A S, et al. Spatiotemporal variance-guided filtering: real-time reconstruction for path-traced global illumination[C]//Proceedings of High Performance Graphics. 2017: 1-10.
- [16] DAMMERTZ H, SEWTZ D, HANIKA J, et al. Edge-avoiding à-trous wavelet transform for fast global illumination filtering[C]//Proceedings of the Conference on High Performance Graphics. 2010: 67-75.
- [17] KARIS B. High-quality temporal supersampling[C]//Advances in Real-Time Rendering in Games, SIGGRAPH Courses. 2014: 1-2.
- [18] SELLERS G, KESSENICH J. Vulkan programming guide: the official guide to learning vulkan[M]. Addison-Wesley Professional, 2016.
- [19] NVIDIA Corporation. Vulkan ray tracing tutorial[EB/OL]. 2024[2024-03-16]. https://github.com/nvpro-samples/vk_raytracing_tutorial_KHR.
- [20] Khronos Group. Vulkan 1.3 specification[M/OL]. 2024[2024-03-16]. <https://registry.khronos.org/vulkan/>.
- [21] 蔡至诚. 混合实时渲染关键技术研究[D]. 杭州: 浙江大学, 2022.